OR Spectrum (2026) 48:541–586
https://doi.org/10.1007/s00291-026-00847-0

ORIGINAL ARTICLE

Efficient insertion and reoptimization algorithms for a customer-oriented dynamic dial-a-ride problem

Arne Schulz[1,2] · **Christian Pfeiffer**[1]

Received: 15 November 2024 / Accepted: 31 December 2025 / Published online: 2 February 2026
© The Author(s) 2026

Abstract

The paper investigates a stochastic-dynamic customer-oriented dial-a-ride problem of a ridepooling provider in a large city. Due to competing mobility services, the ridepooling provider needs to offer the customers short waiting and travel times. New customer requests are inserted to maximize their acceptance probability which depends on the customer's detour in relation to their direct travel time. The trade-off between current and future customers is included by a "potential" that assesses the attractiveness of a tour for future customers. As we need to compute offers quickly, we introduce several precomputation steps, which are done in the meantime between two customer requests, as well as a clustering and a sampling method to reduce the computational effort and use parallelization. These methods allow to solve instances with larger numbers of customers. Our methods are evaluated in a comprehensive computational study with 2700 instances. The main result is that it is best to weight the interests of the current and the future customers equally. Furthermore, the approach can be used by the provider to steer the system's performance. Given a desired service level, i.e. an acceptance rate, the provider can use the approach to decide on the required number of vehicles to reach this service level. Besides, the length of the delivery time windows is important to solve the trade-off between a sufficiently high degree of freedom to reinsert accepted customers in favour of new customers and the system's attractiveness for accepted customers.

Keywords Stochastic-dynamic dial-a-ride problem · Insertion · Lookahead · Acceptance probability · Potential for future customers · Clustering

✉ Arne Schulz
arne.schulz@uni-hamburg.de; arne.schulz@hsu-hh.de

Christian Pfeiffer
christian.pfeiffer@uni-hamburg.de

[1] Institute of Operations Management, Universität Hamburg, Moorweidenstraße 18, 20148 Hamburg, Germany

[2] Institute of Quantitative Logistics, Helmut Schmidt University, Holstenhofweg 85, 22043 Hamburg, Germany

1 Introduction

Although dynamic routing problems have already been investigated for a long time, only a few authors considered the dynamic dial-a-ride problem as a mobility concept for the general public especially in large cities. However, this problem variant is of significant practical relevance, as in recent years several ridepooling providers launched in European cities with the aim to become a viable alternative for end users in comparison with buses, taxis or private cars and to reduce pollution by pooling rides together (see ioki GmbH 2024; MOIA GmbH 2024; Padam Mobility 2024).

As the general public has several options like taxis, public transport or private cars, ridepooling providers have to focus on short waiting and journey times to be competitive. Pfeiffer and Schulz (2022a) investigated the situation of these ridepooling providers in a static setting and introduced an objective function which minimizes the sum of relative detours (waiting and journey time in relation to direct travel time) of all customers. Among others, Vallée et al. (2017, 2019) maximized the number of accepted requests in a dynamic setting. In this paper, we combine both in a stochastic-dynamic setting. First, we maximize the probability that a randomly arriving customer accepts our offer. For this, we assume that the acceptance probability depends on the customer's relative detour. Second, we take into account probabilistic information about future requests to be able to best insert hypothetical future customers and thereby also include the trade-off between current and future customers.

The contribution of this paper is as follows, whereby the main focus is on points 4 and 6:

1. We introduce a stochastic-dynamic dial-a-ride problem maximizing the number of accepted customer requests where customers accept an offer more likely if their relative detour is shorter.
2. We introduce an efficient insertion method for randomly arriving requests by computing feasible reinsertions of already accepted passengers in different tours. Thereby, we use an improved computation for the forward time slack for our setting and an insertion algorithm using the structure of our objective function to minimize the relative detour of the inserted customers.
3. We present a new acceptance function according to which the customers accept an offer based on the relative detour.
4. We implement a new algorithm component (called "potential") taking into account possible future requests in the current choice of how to insert a customer.
5. We implement parallelization as well as a clustering, a sampling and a precomputation method to ensure short response times on customer requests.
6. We evaluate the new methods in a comprehensive computational study and derive managerial insights.

The paper is structured as follows: In Sect. 2, we review the literature before we introduce our problem setting in detail (Sect. 3). Afterwards, we present the basic algorithm in Sect. 4 and the potential calculation in Sect. 5. The solution procedure is tested in a computational study in Sect. 6. The paper closes with a conclusion (Sect. 7).

2 Positioning in the literature

In this section, we discuss the relevant literature about dynamic dial-a-ride problems (DARP) and the related pickup and delivery problem (PDP). Unlike in the static case, the dynamic DARP is generally defined as a problem where some information (such as customer requests) is not known upfront but revealed over time. The provider has to adjust the tours based on the newly revealed information (Ho et al. 2018), typically aiming to accept as many requests as possible (Attanasio et al. 2004). We split up the section into five parts, each focussing on a different aspect of our problem setting or solution approach. The first subsection considers applications of dynamic DARPs, the second stochastic elements, the third demand management, the fourth the modelling of the problem setting, and the fifth focusses on solution approaches. Moreover, we refer to the surveys by Berbeglia et al. (2010), Fleckenstein et al. (2023), Ho et al. (2018) and Soeffker et al. (2022).

2.1 Applications of dynamic DARPs

Historically, dynamic DARP applications have mostly focused on the transport of elderly or disabled passengers (Attanasio et al. 2004; Häll et al. 2015; Jaw et al. 1986; Madsen et al. 1995; Quadrifoglio et al. 2008) as well as patient transportation (Hanne et al. 2009; Souza et al. 2022). These services tend to have two points in common: it is difficult or forbidden to reject customer requests and the users of such a service are often relying on the transport, i.e. have no alternative mobility option.

This is different from ridepooling applications for the general public for which the literature is evolving (Gaul et al. 2021; Vallée et al. 2017, 2019, 2020). In these works, customer rejection is allowed and often necessary to keep a feasible solution. Rejections are also less of an issue, since users will likely have transportation alternatives (e.g. public transport, taxis, private cars)—especially in urban areas where modern mobility options like bike sharing (Brinkmann et al. 2019), car sharing (Illgen and Höck 2019), or scooter sharing (James et al. 2023) have gained in popularity. In our work, we implicitly capture these alternative mobility options by giving the user an acceptance function: If our offer is too poor, the user will be lost and uses a more convenient alternative (e.g. one that offers an earlier arrival at the destination). This means that we only consider the ridepooling provider while not integrating other modes of transportation directly (Liu et al. 2019). If customers decide for another mode of transportation, they simply drop out of our system. A similar system is taxi sharing, which is especially well-known in developing countries (Hosni et al. 2014).

2.2 Stochastic elements in dynamic DARPs

Stochastic elements, like our acceptance function, have been included in dynamic DARPs as well. Swihart and Papastavrou (1999) considered a single-vehicle PDP where requests arrive according to a Poisson process. They minimized the time the requests spend in the system and utilized queuing theory to investigate the problem. In the paper at hand, customer requests are also assumed to arrive according to a Poisson distribution.

In Xiang et al. (2008), the authors considered several stochastic events: Travel and service times are uncertain, customers might not show up and the vehicle might get stuck in traffic jams or break down entirely. In Schilde et al. (2011), the authors considered stochastic return transports: With a certain probability, a serviced request might lead to another return request later in the day. Wang et al. (2019) investigated customer cancellation behaviour. In our paper, we assume that customers never cancel a trip which they have already accepted. Nevertheless, they can reject an offer if it is not attractive enough.

A further aspect of stochasticity can be included by the drivers who might reject customer requests as well if they work on their own and only cooperate with a platform assigning them trips. We assume in our setting that drivers are employed at the ridepooling provider and therefore fulfill all requests in the sequence the provider determines.

2.3 Demand Management

In this section, we focus on dynamic occurrence of customer requests. When customer requests become known over time, the decision maker needs to decide on routes while not all customer requests are known. This results in an evolving system where routes are updated whenever new requests occur. Moreover, inserting a customer now blocks capacity for future customers (opportunity costs) such that rejecting the current customer can lead to an overall better solution. The decision on acceptance or rejection and the concrete offer to a customer is known as demand management. In this section, we focus on demand management in dynamic problem settings [for demand management in static dial-a-ride problems we refer to Anzenhofer et al. (2025)].

The provider can manage demand by designing the offer itself as well as by selecting the price (Waßmuth et al. 2023). Our focus is on the offer. The recent paper by Haferkamp (2024) claims to be the first to adopt the idea of offerings for dynamic vehicle routing. By the offer selection, the operator can steer demand temporally as well as geographically (Fleckenstein et al. 2023). The provider can, for example, offer the same ride for a later point in time to stretch demand fulfillment or provide more attractive offers in a certain part of the service area, e.g. to balance the demand over the service area. Typically demand management aims at maximizing the profitability by maximizing revenue, i.e. prioritizing high-value customers, or by maximizing utilization, i.e. maximizing the number of accepted requests (Waßmuth et al. 2023). As we consider a ridepooling provider which might work as part of the service to the public, we aim at minimizing the number of rejected requests by computing offers that fit well into routes. Our potential method evaluates the consequences of an insert for the next occurring customer, i.e. the displacement cost of the current insert in form of displaced future customers. Thereby, customers with certain characteristics should not be disadvantaged by the approach (we evaluate this aspect in Sect. 6.3.4).

Our system can be classified as follows [in brackets classification key words in Fleckenstein et al. (2023)]:

- Customer requests comprise a pickup and a delivery location (point-to-point).

- We minimize the number of rejected requests (max. number of orders).
- Offers are computed according to the possibility to insert them in the current tours (availability).
- As customers order their trip when they want to start, our system needs to response almost immediately (real time).
- Booking and service horizon are overlapping. The provider needs to response in real time on a customer request, which includes a demand management and routing decision.
- Accepted customers' service must not deteriorate too much. Hence, there are side constraints restricting the time in the system of accepted customers (waiting and travel time).

This is exactly the classification of a mobility-on-demand service in Fleckenstein et al. (2023) (Table 2 in their paper). Note that a 24/7 ridepooling service could also be interpreted to have an "infinite" service horizon. However, ridepooling operators typically have service times including closing hours in practice (Berthold et al. 2025).

2.4 Modelling

Greedily deciding on a customer's request without including future customers is very fast but might worsen the overall solution quality while including future demand into the current decision may add complexity to the solution approach (Shah et al. 2020). Thus, a solution policy is used to solve the trade-off between speed and including future demand reasonably. The literature offers several approaches with different degrees of complexity. Future demand can simply be predicted from historical data (Alonso-Mora et al. 2017b) or dynamic lookahead policies simulate future demand over a predefined time horizon (Brinkmann et al. 2019). Thereby, the simulation also evaluates opportunity costs of the current insertion. In our approach, we determine the expected opportunity costs for the next customer's insert by evaluating all possible requests which can occur next.

Powell (2022) stated that a stochastic lookahead policy requires to solve a stochastic lookahead model with strong approximations and the complexity of stochastic lookaheads is a reason why deterministic lookaheads are the widely used lookahead policy in practice. Moreover, he defined six classes of approximations: horizon truncation, outcome aggregation or sampling, discretization (time, states, decisions), stage aggregation, latent variables (ignoring variables) and policy approximation. In this paper, we have a fixed set of possible stops for pickup and delivery locations, which allows us to evaluate all possible future requests that might occur next (horizon truncation), but we approximate tours by only allowing to reinsert already scheduled customers (policy approximation). However, given this restriction, the expected probability that the next customer rejects is calculated exactly. In further evaluations, we additionally introduce a clustering method (state aggregation; Appendix G) and evaluate our potential method only for a subset of all possible future requests (outcome aggregation or sampling; Sect. 6.3.7).

As we fully evaluate all options (given the restriction in tour optimization) for the next customer but none for later customers, we have a very detailed evaluation over a

rather short time horizon like in a cost function approximation (Soeffker et al. 2022). In contrast to other approaches (e.g. Haferkamp 2024), we compute opportunity costs directly in the expected loss of revenue for the next customer instead of a tour oriented cost measure (e.g. longer trip duration due to the new customer).

A further method is value function approximation which captures a longer time horizon but evaluates only a fraction of possible decisions (Soeffker et al. 2022; Ulmer and Thomas 2020). In this paper, we assume a revenue of one for every accepted customer. Minimizing the rejection probability for the next customer also maximizes the probability of the complementary event that the next customer accepts which is identical to the expected revenue generated by the next customer. Thus, our cost function directly corresponds to the value function which maximizes the expected reward of the next customer (see end of Sect. 3.2). For a more detailed overview of possible methods we refer to Soeffker et al. (2022).

Focussing more closely on dynamic ridepooling problems, the paper by Haferkamp (2024) shares substantial relations with our problem setting. Haferkamp (2024) uses a cost function approximation to balance routing costs and acceptance probabilities. The author computes the best insert for every possible discrete pickup time and constructs the best offer from them in the sense that the weighted difference between acceptance probability and routing effort is maximized. Thereby, the current customer's interest (acceptance probability) and the future customers' interests (routing effort) are included. In our paper, we follow the same objective but balance the current customer's acceptance probability with the acceptance probability of the next occurring customer. A further difference is that the customer gets several options offered in Haferkamp (2024) and can choose either one of them or cancel the request. Although ridepooling providers like MOIA also offer several transportation options to their customers in practice, we restrict ourselves to offer the customer only a single transportation option. This design choice is due to our assumption that the customers want to reach their destination as fast as possible. Thus, they would always choose the request with the earliest arrival time. This is also in line with Sayarshad and Gao (2018) and Liu et al. (2019) who consider competing ridesharing operators and alternative means of transportation, respectively. Both assume that minimizing waiting time is the deciding factor for the customer's choice. The approach by Haferkamp (2024) ensures an offer in the very near future by a given maximum lead time between request and pickup of the customer. Another difference is how routes can be adapted after the acceptance of the customer. Both papers, Haferkamp (2024) and ours, restrict the time that an accepted customer's arrival time is allowed to be delayed by after acceptance. From the practical perspective there must be a restriction such that the customer can rely on the offer and knows its conditions upfront, as unexpected delays are rated more negatively (Geržinič et al. 2023). Our paper includes this aspect by a customer independent time span, which is communicated to the customer and by which we are allowed to delay the customer's arrival time at their delivery location. Haferkamp (2024) uses a function that depends on the direct travel time of a customer to compute the tolerable delay. Both, Haferkamp (2024) and we, aim at minimizing the number of rejected customer requests.

Mostly customer requests are integrated into tours whenever they occur. However, some papers integrate dynamic customer requests in batches. In Agussurja et al.

(2019), the authors assume already that requests occur in batches. If a certain time interval is waited for new requests, requests are assigned in a rolling horizon framework (Gaul et al. 2021, 2025). Here none, one or several customers can arrive before all of them are assigned to the tours.

Our investigated setting differs in several further ways from other settings investigated in the literature. First, we do not differentiate between waiting and driving time. We assume that the customers' acceptance decisions depend completely on the time the customers reach their destinations, independent on the distribution in waiting and driving time, leading to a degree of freedom in deciding on the customers' pickup times. It might be that a later pickup time leads to a better matching with a vehicle and results in an earlier arrival time at the delivery location (Ke et al. 2020). Nevertheless, minimizing the time between request and arrival at the delivery location is a short time horizon. Same day delivery with a time horizon of up to a few hours is considered in Klein and Steinhardt (2023) and Ulmer (2020) while, according to an analysis in Haferkamp (2024), most of the literature on dynamic vehicle routing focuses on time window management for the next day(s). Second, we do not include pricing in our approach (Ma et al. 2022). Pricing can influence the customers' orders and thereby the system's performance (Anzenhofer et al. 2024). For example, a less attractive offer, i.e. with a later arrival time at the delivery location, can be combined with a cheaper price to increase the acceptance probability. Third, our setting always allows to add a new customer at the end of a tour. Thus, there is always a feasible offer for a customer. Rejection occurs if the customers assess their offer as too unattractive. In the literature, service often needs to happen within time windows (Avraham and Raviv 2021; Hosni et al. 2014; Hungerländer et al. 2017; Klein et al. 2019; Koch and Klein 2020; Lang et al. 2021; Mackert 2019; Xiang et al. 2008; Yang et al. 2016). Fourth, we do not consider relocation of idle vehicles (Alonso-Mora et al. 2017a; Schmid 2012). Vehicles simply stay where they are whenever they do not serve customers or are on the way to serve customers.

2.5 Solution approaches

In the following, we present several papers that share some similarities and differences to parts of our solution approach, especially the insertion and reinsertion algorithms and the maximization of accepted requests: Vallée et al. (2017, 2019, 2020) presented several articles investigating a real life ridepooling provider. In Vallée et al. (2019), a reinsertion algorithm tries to move existing customers around to make room for the new customer [similar to the rejected-reinsertion operator in Luo and Schonfeld (2011)], whenever a new customer request cannot be feasibly inserted. This approach shares similarities to what we do in the precomputation phase. However, their reinsertion algorithm is based on Large Neighborhood Search while we simply remove a customer from a route and reinsert them into another. In either case, they do not include information about potential future demand which is central in our approach.

Coslovich et al. (2006) employed a two-phase algorithm. In the time between two customer requests, they precalculate certain neighborhoods of the current solution. When a new customer arrives into the system, they then utilize these neighborhoods

to improve the ability to accept the customer. This approach is very similar to our precomputation phase, as we are effectively also exploring neighborhood solutions which are then used when a new customer request appears. Unlike our procedure, the customer to tour assignment in Coslovich et al. (2006) is fixed and cannot be changed in a neighborhood solution. Instead, the neighborhood for each tour consists of all feasible tours that can be reached by a 2-opt move from the current tour.

Two papers that address a similar issue to our potential calculation by including information about future requests are Ichoua et al. (2006) and Tafreshian et al. (2021). In Ichoua et al. (2006), dummy customers are inserted into the system to keep vehicles in certain locations where demand is predicted to appear in the future. In Tafreshian et al. (2021), the authors examined an application with ridepooling for the general public. They employed an offline phase to compute good combinations of users using historical data. The potential calculation has also similarities to a rollout algorithm, as it approximates the reward of the current insert by including a factor for the next insert (Goodson et al. 2013), i.e. over a very limited time horizon. However, in rollout algorithms typically only a few candidate decisions are evaluated (Soeffker et al. 2022). Although we calculate the evaluation of the next insert optimally (given that accepted customers can only be reinserted), the procedure is clearly heuristic, since it does not concretely consider all future customers beside the next one.

Our potential calculation includes the evaluation of all possible requests which can occur next. With an increasing number of possible stops, this number increases quadratically such that at some point not all possible future requests can be evaluated any more. To address this issue, we present a parallelization and a clustering method. Moreover, we use a sampling method to evaluate only a subset of possible requests. Sampling has also been used in the literature. Heitmann et al. (2023) introduced a multiple scenario approach which samples the routing anticipating future requests.

3 Problem description

The section starts with a verbal problem description in Sect. 3.1. Afterwards, we present a sequential decision process in Sect. 3.2.

3.1 Verbal problem description

We consider a dynamic dial-a-ride problem with $|K|$ homogeneous vehicles with capacity Q. Each vehicle drives a single tour $k \in \{1, \ldots, |K|\}$. Let n be the total number of requests, $P = \{1^+, \ldots, n^+\}$ the set of all pickup locations, and $D = \{1^-, \ldots, n^-\}$ the set of all delivery locations. We assume that the pickup and the delivery location for every customer is one of V given stops. The stops can be interpreted as the nodes of a complete graph such that customer requests form a trip from one node to another. Let I be the set of these V stops. In practice, these stops can be real stops, like public transport stations, virtual stops or represent zones (see e.g. Ackermann and Rieck 2023; Fagnant and Kockelman 2014). Virtual stops can be understood to be real stops whose position cannot be recognized in the streetscape. The customer only gets to know via a smartphone application where to enter the vehicle. This can

be, e.g., a lay-by or simply at the curbstone close to a certain address. Therefore, a large number of virtual stops can be implemented without any constructional changes in the streetscape. In practice, some providers like MOIA GmbH (2024) use virtual stops in ridepooling services.

Requests $i \in \{1, \ldots, n\}$ arise over time. Their details are not known beforehand. Request i comprises a pickup location i^+ and a delivery location i^- as well as a demand q_{i^+}, i.e. the number of people who want to travel together. Let $t_{jj'}$ be the travel time from location j to location j' for every pair $j, j' \in I$. Furthermore, let $t_i = t_{i^+,i^-}$, i.e. the direct travel time between customer i's pickup and delivery location. We assume that all requests $i \in \{1, \ldots, n\}$ wish to be transported as soon as possible after their request gets known at time e_{i^+}. The task of our solution approach is to assign the new dynamic request to one of the $|K|$ tours such that both, pickup and delivery location, are assigned to the same tour and the vehicle visits for each customer first the pickup and then the delivery location. We assume the triangle inequality to hold and that tours are only changed when the vehicle is at a customer location. This means that the vehicle never changes its route while it is driving to a location, but the tour after the next location can be changed.

The primary objective is to maximize the number of accepted requests which are inserted into tours. We assume that customers will accept the proposed insert with a certain probability which is dependent on the relative detour they would incur (compare Pfeiffer and Schulz 2022a; Schulz and Pfeiffer 2024b). The relative detour (*RD*) for a customer i is measured as

$$RD = \frac{B_{i^-} - e_{i^+} - t_i}{t_i}, \tag{1}$$

where B_{i^-} is the arrival time at delivery location i^- of request i. (1) measures the relative detour of a customer i, i.e. the time between their request e_{i^+} and their arrival at the delivery location B_{i^-} minus their direct travel time t_i in relation to their direct travel time t_i. An example for (1) is given in Pfeiffer and Schulz (2022a). The acceptance probability for customer i is then calculated by

$$e^{-\mu \cdot RD} \tag{2}$$

where $\mu > 0$ is an additional weight parameter. An earlier proposed arrival time at the customer's delivery location will therefore increase the probability of acceptance. A customer that does not accept our offer is considered as rejected.

Although we assume that customers who accepted an offer never reject it later, integration of new dynamic requests should not increase *RD* arbitrarily for another already accepted customer i. Therefore, we limit the maximum time shift in the arrival time at delivery location B_{i^-} to a later point in time by l. We calculate $l_{i^-} = B_{i^-} + l$ directly after customer i accepted our offer. Together with the request time e_{i^+}, customer i has to be transported from their pickup to their delivery location within the time window $[e_{i^+}, l_{i^-}]$. The value of l can be communicated to the customer beforehand and is thus included in the acceptance probability. Note that this means that customers can never be rejected after acceptance, which is in line with the findings

by Geržinič et al. (2023), who found that a canceled service after booking results in significant disutility for the customers.

Figure 1 shows the situation for a customer i. The customer arrives at point in time e_{i^+} in the system while the vehicle's schedule is to drive $A \to B \to C \to D$. Then, customer i is included in the tour by visiting i^+ between locations B and C and i^- between locations C and D. With the new tour schedule, customer i's arrival time B_{i^-} at the delivery location is known such that the customer can decide on the offer according to (1) and (2). After acceptance the customer can be delayed because of new arriving customers (like locations C and D in the figure due to customer i). However, $l_{i^-} = B_{i^-} + l$ sets the deadline for any delay of customer i such that customer i is certainly served within the time interval $[e_{i^+}, l_{i^-}]$. The notation used in this paper is summarized in Table 6 in Appendix A.

In line with the literature (Sheridan et al. 2013; Tafreshian et al. 2021), we assume the interarrival times between two consecutive customer requests to be distributed according to an exponential distribution, though our presented algorithm would also work with any other probability distribution.

3.2 Sequential decision process

The problem can be modelled as a sequential route-based Markov decision process (Powell 2022; Ulmer et al. 2020)

$$(S_0, W_1, x_1, \overline{W}_1, S_1, W_2, x_2, \overline{W}_2, ..., S_{n-1}, W_n, x_n, \overline{W}_n, S_n) \tag{3}$$

with

- decision epochs $i = 1, ..., n$ which occur whenever a new request of a customer is received with $e_{i^+} < e_{j^+}$ for $i < j$.
- state variables S_0 indicating the initial state that all vehicles are at point in time 0 at the depot and S_i picturing the state after customer i decided on our offer for the trip. S_i, $i = 1, .., n$, includes all information on the current state of the system. This includes the request time e_{i^+}, the number of customers for each so far known request and for every vehicle k the current position p_k^i as well as the sequence of customer locations s_k^i to be visited in the tour. As an example $s_k^i = (1^+, 2^+, 1^-, 2^-)$ illustrates that vehicle k picks up customer 1 next before customer 2 is picked up and customer 1 is delivered afterwards before customer

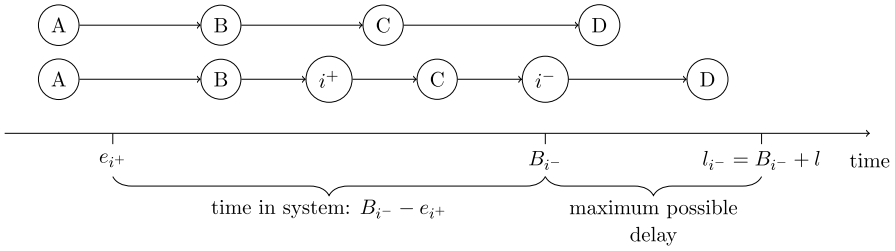


Fig. 1 Overview of situation for customer i

2 is. This means that we know for every entry of s_k^i the physical location as well as the associated customer request. The current position p_k^i is given by coordinates in the service area and the remaining travel time to reach the first location in s_k^i. If s_k^i contains at least one location, we know that the vehicle is driving on the direct way from its current position to the first location in s_k^i. If s_k^i is empty, p_k^i corresponds to the previous customer location the vehicle has visited and the remaining travel time to the next location is 0. Thus, there is no vehicle relocation. Moreover, $q^i = (q_{1^+}, ..., q_{i^+})$ contains the number of customers in the so far known requests. Note that those who are served (delivery location has been visited) can be removed from q^i. Together, $S_i = (e_{i^+}, q^i, p_1^i, s_1^i, ..., p_{|K|}^i, s_{|K|}^i)$.

- decision variables x_i, $i = 1, ..., n$. Variable x_i pictures our offer for customer *i*. The objective function (1) only depends on the arrival time B_{i^-} of customer *i* at their delivery location. Moreover, the maximum time shift in the arrival time *l* at the delivery location is relevant for the customer to decide on the attractiveness of the offered ride. Thus, the offer contains the scheduled arrival time B_{i^-} as well as the latest possible arrival time l_{i^-}. Furthermore, our decision includes the vehicle *k*, the location P_i^- visited before the pickup of the new customer is visited, and the location D_i^- visited before the delivery location of the new customer is visited, i.e. $x_i = (B_{i^-}, l_{i^-}, k, P_i^-, D_i^-)$. Only B_{i^-} and l_{i^-} are communicated to the customer. Note that offer x_i needs to be feasible. Thus, s_k^i needs to be a feasible tour when adding location i^+ after location P_i^- and location i^- after D_i^- in vehicle *k*'s tour. The tour is feasible if i^+ is visited before i^- (holds for all earlier customers because it was checked when inserting them into s_k^i), no delivery location j^- is reached later than the latest possible arrival time l_{j^-} for the corresponding customer *j* and the vehicle capacity *Q* is never exceeded during the trip. Note that we do not consider service times (can be assumed to be part of the travel time). Furthermore, it is never useful to wait, as all customers want to be delivered as early as possible and all inserted customers are available (insertion not before their request time). Thus, the arrival time at a location is the arrival time at the previous location plus the direct travel time between the previous location and the considered location.
- exogenous information variables W_i and $\overline{W}_i$, $i = 1, ..., n$. Exogenous information variables represent the stochastic influences by customer request *i* (W_i) and the stochastic acceptance decision of customer *i* ($\overline{W}_i$) according to (2). $W_i = (i^+, i^-, e_{i^+}, q_{i^+})$, i.e. it comprises the pickup stop i^+, the delivery stop i^-, the request time e_{i^+}, and the number of persons q_{i^+} of request *i*. $\overline{W}_i$ is 1 if customer *i* accepts the offer and 0 otherwise.
- the cost function: If the customer rejects, we realize costs of 1 for the rejection which we want to minimize. If the customer accepts, there are neither costs nor is there any reward.

The transition function $S_i = S^M(S_{i-1}, W_i, x_i, \overline{W}_i)$ determines state variable S_i given the former state S_{i-1}, customer *i*'s characteristics W_i, our offer x_i, and the customer's decision $\overline{W}_i$ as follows:

- Tour changes:
  - If the customer accepts, offer x_i is realized and the customer is inserted into tour s_k^{i-1} by adding its pickup location i^+ directly after location P_i^- and its delivery location i^- directly after location D_i^-. Let $\bar{s}_k^i$ be the resulting tour. Then, time proceeds until e_{i^+}, i.e. the vehicle drives its tour without any waiting until e_{i^+} such that s_k^i corresponds to $\bar{s}_k^i$ without those customer locations visited in the meantime. Note that all customer locations are at the V virtual stops. Thus, visited locations in the tour as well as travel times $t_{jj'}$ between all virtual stops j and j' are sufficient to compute arrival times for all locations.
  - If the customer rejects the request, tours are processed accordingly such that s_k^i corresponds to s_k^{i-1} without those customer locations visited in the meantime.
  - All other tours are processed in the same way as tour k if customer i rejects the offer.

- The current point in time is e_{i^+}.
- p_k^i corresponds for every k to the physical location where the vehicle is at point in time e_{i^+} when following tour s_k^i without any waiting (it stays at the last location if there is no further one) and the remaining travel time to the next location (if there is one).
- q^i corresponds to q^{i-1} extended by q_{i^+} if i accepts. Otherwise, q^i corresponds to q^{i-1}.

The transition function shows the challenges of the problem:

(C1)the future is uncertain because of unknown customer requests and their stochastic acceptance decision.
(C2)due to the large number of possible customer requests, there is a significant number of possible states.
(C3)the decision space's size, including all feasible inserts of a customer, is also significant while computation times need to be very short.

In contrast to most sequential decision processes (Powell 2019, 2022), our reward $C(S_{i-1}, W_i, x_i, \overline{W}_i)$ does not directly depend on the current state variable and decision variable but on the exogenous information variables $\overline{W}_i$. However, $\overline{W}_i$ depends on x_i and x_i depends on S_{i-1} and W_i. It holds $C(S_{i-1}, W_i, x_i, \overline{W}_i) = 1 - \overline{W}_i$. This means that the contribution is 0 if the customers accept and 1 if they reject. The overall objective to find a policy π, determining decisions x_i^π to minimize the expected rejection rate, is

$$\min_\pi \mathbb{E}\left[\sum_{i=1}^{n} C(S_{i-1}, W_i, x_i^\pi, \overline{W}_i)\right]. \tag{4}$$

Note that in the standard form of a sequential decision process the contribution is maximized (Powell 2019). This can simply be realized by setting $\overline{C}(S_{i-1}, W_i, x_i^\pi, \overline{W}_i) = 1 - C(S_{i-1}, W_i, x_i^\pi, \overline{W}_i) = \overline{W}_i$ whereat $\overline{W}_i$ is the reward for an accepted customer request as it is done in Ulmer et al. (2019). Then, the objective is $\max_\pi \mathbb{E}\left[\sum_{i=1}^n \overline{C}(S_{i-1}, W_i, x_i^\pi, \overline{W}_i)\right]$.

Note further that the expected value in (4) depends on the initial state S_0. As we assume the initial state to be constant in this paper (all vehicles start in a common depot), we omit the condition. In the following sections, we explain our policy π.

4 Insertion and precomputation algorithm

Figure 2 pictures the decision process in (3) and shows the first part of our method to deal with the problem's challenges. We deal with the uncertainty of customer requests (C1) by reacting on them when they are realized (after W_i). Whenever a new customer request emerges, we find the best insert for this customer (Sect. 4.1), for now without the potential method, i.e. our policy minimizes the chance that the current customer *i* rejects:

$$\min_\pi \mathbb{E}\left[C(S_{i-1}, W_i, x_i^\pi, \overline{W}_i)\right].$$

Next, the customer decides on the offer ($\overline{W}_i$). With the decision, we know whether the customer accepts the offer or not (C1). If the customer accepts, they are scheduled according to the offer resulting in state S_i. If not, time simply proceeds between S_{i-1} and S_i.

Now, we are aware of the current state of the system (C2) and have time to prepare for the next customer request. We use this time to precompute possible reinserts of customers in different tours to allow for better inserts for the next customer (C3). As the interarrival time is uncertain (C1), this is run until either the next customer arrives or until all possible precomputation inserts have been calculated. In the first case, only those reinserts can be used which have already been evaluated. In Appendix D, we present a method to parallelize the insertion and the precomputation procedure ((C1) and (C2)).

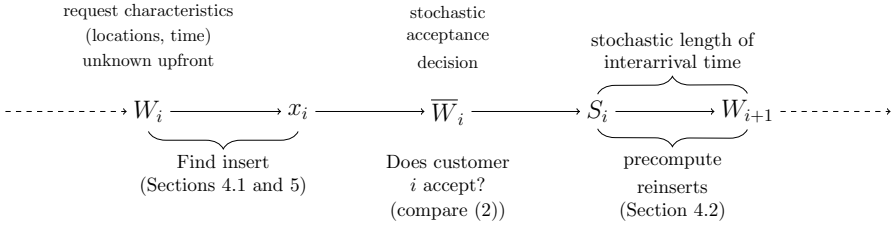


Fig. 2 Overview of the decision process

4.1 Insertion procedure

We explain the insertion of a new customer in this section. Thereby, we use the Forward time slack (FTS) which is explained in detail in Appendix B. We use two insertion possibilities: First, we check whether the new customer can be inserted into a tour $k \in K$ directly. Second, we check whether the new customer can be inserted into a tour $k \in K$ if another customer is shifted from the tour k into a different tour k'. In order to do this quickly, we use the time between two consecutive requests to precompute possible insertions of customers into different tours. The set of precomputed tours where one customer is missing is denoted by K'. Let further $K'_k \subseteq K'$ be the subset of precomputed tours that were generated by removing a customer from tour $k \in K$. The insertion procedure will therefore consider an insert into every tour $k \in K \cup K'$. In the simplest case, we select the insert that maximizes the acceptance probability (2) for the new customer. In Sect. 5, we show how to incorporate additional information into the evaluation.

The insertion procedure itself is rather straightforward. We present a detailed explanation in Appendix C.

4.2 Precomputation of new tours for insertion

Removing a single already accepted customer and reinsert them into another tour leads to a higher degree of freedom for the insertion of new requests. If a customer is reinserted in a different tour, it is more likely that a new customer can be inserted in the former tour of the reinserted customer. The insertion of the new customer in the tour without a formerly scheduled customer can be done directly with Algorithm 5 (Appendix C). However, Algorithm 5 can also be used to reinsert customers in different tours. After a customer has been inserted and the system is "idle" until the next incoming dynamic customer, all possible reinsertions of single customers into other tours are precomputed. The precomputation is stopped if the next dynamic customer arrives before it is fully done. The already calculated reinsertions can still be used for the next insert in such a case.

In order to accomplish this, every time the precomputation phase is executed, it is checked for each customer of every tour whether there is a feasible insertion possibility of that customer into every of the other tours. Whether an insert exists can be checked with Algorithm 5 (Appendix C). In doing so, the time window of the considered customer has to be respected. This can be reached by setting $o = l_{i^-} + 1$ in Line 2 of the algorithm. T is the current point in time and the request is already known; thus, T is later than the request time of the considered customer. If an insertion is possible, we save it as a possible reinsertion. Hence, all possible reinsertions can be evaluated by repeating the described procedure for all customers in all tours.

The computational effort to evaluate all inserts for a customer into a tour k of length $\bar{n}_k$ is bounded by $\mathcal{O}(\bar{n}_k^2)$. Since there are n customers, this results in $\mathcal{O}(n \cdot \bar{n}_k^2)$ as an upper bound on the computational effort to evaluate all possible reinsertions of customers in tour k. As $\sum_{k=1}^{|K|} \bar{n}_k \leq 2n$, the computational effort to evaluate all possible reinsertions in all tours is $\mathcal{O}(n^3)$ in the worst case.

After a dynamic customer has been inserted, the previously precomputed reinsertions do not all need to be discarded. If the corresponding tours to reinsert have not been changed by the dynamic insert and the current time still allows for the possible route change (i.e. the customer in question has not been and would not have been picked up yet), the reinsertion is still valid.

5 Including the potential of tours for other customers

In this section, we extend our approach from Sect. 4 by including opportunity costs for customer $i+1$ into the decision on the offer for customer i. Thus, our policy minimizes not only the expected rejection rate of customer i but also of customer $i+1$ afterwards:

$$\min_{\pi}(1-\omega)\cdot\mathbb{E}\left[C(S_{i-1},W_i,x_i^{\pi},\overline{W}_i)\right]+\omega\cdot\mathbb{E}\left[C(S_i,W_{i+1},x_{i+1}^{\pi},\overline{W}_{i+1})|\overline{W}_i=1\right]$$

whereat S_i is the system's state resulting from state S_{i-1} if customer i accepts offer x_i^{π} ($\overline{W}_i=1$) and ω is the weighting factor deciding on the weight of the next customer's acceptance probability in comparison to the current customer's acceptance probability. To do so, we use the time between S_i and W_{i+1} to precompute inserts for all possible customer requests $i+1$ (Sect. 5.2). This is possible since we consider certain (virtual) stops as the only nodes where customers can be picked up or dropped off as it is done by many ridepooling providers in practice.

Because of the virtual stops, we can define all possible pickup/delivery relations. If there are V stops, there are $V(V-1)$ possible pickup-delivery relations which are all added to the set of possible customer requests PC. Note that these requests do not contain a request time but only focus on the geographical data. In the simplest case, each of the requests $b\in PC$ is assumed to be equally likely to occur (with probability $1/|PC|$). Should historical data for the demand be given, that can also be included, e.g. like in Núñez et al. (2014), to determine the probability of each of the possible customer requests to occur next.

The idea of our procedure is to use the time between S_i and W_{i+1} (compare Fig. 2) to precompute inserts for all possible customer requests PC. By this, we can evaluate the effect of offer x_i for customer i on the best inserts for customer $i+1$ without knowing its concrete characteristics, i.e. without knowing W_{i+1}. Thereby, we can address (C1) in a more sophisticated way and solve the trade-off between the interests of customers i and $i+1$ (opportunity costs of inserting customer i). Moreover, we know already the best insert for customer $i+1$ if it occurs and thereby also address (C3) and are able to immediately respond on the request of customer $i+1$.

Note that the potential only focusses on the single next request at first glance. However, the time horizon for the next request is not restricted. Thus, any possible request occurring at any future point in time is included in the potential evaluation. The only assumption is that it is the next request. This assumption influences the probability, as for an appearance later in the future the probability is small (because it would imply that no other request occurs in the meantime). Nevertheless, all pos-

sible future requests at any time are included and thereby we have also included later requests beside the next one. Only their probability of occurrence is approximated by assuming that it is the next one. Yet, this approximation is useful because the influence of requests in the later future is smaller than of those in the nearby future.

Although there is only a quadratic number of possible customer requests $i + 1\ (V \cdot (V - 1))$, this number can still be very large. Therefore, we present a parallelization of our method in Appendix F and a method to cluster the virtual stops in Appendix G. Before describing the details in Sect. 5.2, we start with a general overview of the procedure of the potential in Sect. 5.1.

5.1 General overview of potential

Assume a new customer arrives and we have the option of inserting them into one of two tours. If one tour is more attractive to our possible customers, it might incentivize us to keep that tour intact and insert the current customer into the other one. In order to measure this attractiveness, we define the potential of a set of tours $\bar{K} \subset K$ for a customer $b \in PC$ as $PT_{b\bar{K}}$. The potential gives the expected acceptance probability for a customer of type b if they were inserted optimally into the best tour $k \in \bar{K}$. To determine the expected acceptance probability we weight the acceptance probability with the probability distribution for the interarrival times, i.e. how likely it is that the customer will appear in a given time interval. Whenever we evaluate the dynamic insert of a new customer into a tour $k \in K \cup K'$, we consider both the acceptance probability of the dynamic customer for the insert as well as the loss in potential for all of the possible customers $b \in PC$.

Figure 3 gives a general overview of our algorithm if we include the potential. Whenever a new customer arrives into the system, we try to insert them as quickly as possible (see Sect. 4.1), potentially canceling active tasks that were worked on before. Since the actual insertion is very fast, due to the amount of precomputations in Sect. 4.2, the chance of a customer arriving while we still insert the previous customer is negligible. If the customer accepts the insert, the current solution is updated and the precomputation of new inserts is run again. If the customer did not accept, this step can mostly be skipped, since the previous precomputations are still valid. The exception to this are precomputation inserts that are no longer possible due to the advanced point in time. Next, the potential is calculated in several steps. Since the potential is dependent on the current point in time, these steps are run through regardless of whether the solution changed or not. However, some of the information from previous iterations can still be used but might need to be updated according to the new current point in time. An iteration ends if a new customer arrives or if the potential for all inserts has been computed, depending on what occurs first. In the case of an early cancellation, already calculated potential values are used normally while all not calculated potential values are treated as zero.

5.2 Calculation of the potential

In the following, the individual steps on how the potential is calculated and utilized are explained. The potential exploits the fact that there are certain intervals of time

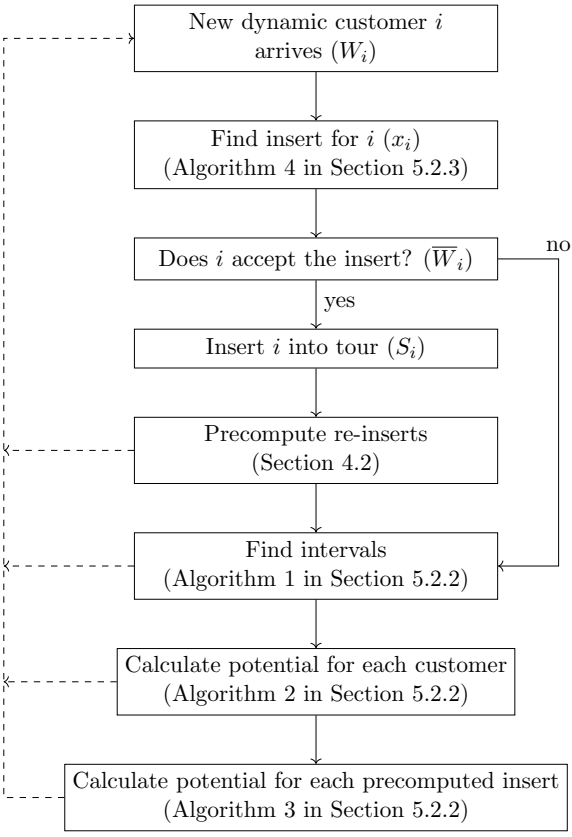


Fig. 3 Overview of our algorithm with potential. The dashed arrows signify that the procedure jumps back to the beginning early if a new dynamic customer request arrives

in which a possible customer's optimal insertion positions remain the same as long as the customer places their request in the interval (Sect. 5.2.1). With this, we can then calculate the best tour at any given point in time for each subset of tours and this possible customer (Sect. 5.2.2). Finally, we can utilize this information to more accurately make a decision for the next real customer (Sect. 5.2.3).

5.2.1 Determining intervals

First, we need to know for each possible customer $b \in PC$ what the intervals are for each tour $k \in K \cup K'$ given we are in point in time T. A time interval $(a_1, a_2]$ means that if customer b places their request during the time interval, the optimal insertion position (maximizing customer b's acceptance probability) remains the same. This is accomplished by Algorithm 1. This algorithm takes advantage of the fact that our vehicles never change their route if they are driving from one location to another. This means that inserting the pickup location b^+ of a customer b after a location in the tour σ_i is only possible if the arrival of the customer into the system happens before we leave σ_i, i.e. $e_b \leq B_{\sigma_i}$.

1: **function** CALCULATEINTERVALS(tour $k \in K \cup K'$, possible customer $b \in PC$, current point in time T)
2: $\varepsilon > 0 \leftarrow$ small number
3: $\sigma_1, \sigma_2 \ldots, \sigma_{\bar{n}} \leftarrow$ current sequence of locations in tour k
4: $T_1 \leftarrow T$
5: **while** $T_1 < B_{\bar{n}}$ **do**
6: Assume customer b arrives at $T_1 + \varepsilon$ and find their best insert into tour k with Algorithm 5
7: $\sigma_i \leftarrow$ location before the pickup location b^+ in that insert
8: $T_2 \leftarrow B_{\sigma_i}$
9: For an arrival in the interval $(T_1, T_2]$, the above insert is always optimal. Save this interval and its accompanying optimal insert's arrival time at the delivery location.
10: $T_1 \leftarrow T_2$
11: **end while**
12: Add the last interval $(B_{\sigma_{\bar{n}}}, \infty)$ with constant optimal acceptance probability of $\exp(-\mu \cdot t_{\sigma_{\bar{n}}, b^+}/t_b)$.
13: **end function**

Algorithm 1 Precomputation of intervals

On the other hand, an optimal insertion position for a request time e_b stays optimal even if e_b is increased, as long as e_b is not greater than the departure time of the location preceding the optimal pickup location of b^+. Thus, the algorithm starts at the current time T and then iteratively calculates all relevant interval boundaries and their respective optimal insertion positions by setting the upper bound of an interval to the departure time at the location preceding the current optimal insertion location of the pickup.

Note that for the last interval (Line 12), the acceptance probability is constant. This assumes that our vehicle stays at the last visited location if no further customers are planned. Thus, if the request time of the possible customer b is greater than the arrival time at the last location in the tour $B_{\sigma_{\bar{n}}}$, it is optimal to go straight to the pickup and then to the delivery location of the possible customer which results in a nominal detour of the travel time between $\sigma_{\bar{n}}$ and b^+, i.e. $t_{\sigma_{\bar{n}}, b^+}$. While we save the acceptance probability for the last insert, for all other inserts we instead save the arrival time at the delivery location. This is because for all inserts preceding the last one, the arrival time is a constant value, but the acceptance probability still depends on the actual arrival of the possible customer in the system. This flips for the last insert where the arrival time at the delivery location is not constant, but the acceptance probability is.

An example for the functionality of Algorithm 1 can be found in Appendix E.

5.2.2 Determining best tours for each possible customer

Once we have run through Algorithm 1 and determined all relevant interval boundaries for the current point in time T, each possible customer $b \in PC$ and each tour $k \in K \cup K'$, we need to combine the intervals for each customer $b \in PC$ across the tours. However, the different intervals are not equally likely to occur which should be taken into account in the calculations. The acceptance probability for interval $(a_1, a_2]$ needs to be weighted with the probability that customer b actually places their request in that interval. In this paper, we assume an exponential distribution of the interarrival

times with an expected value of $1/\lambda$, though other probability distributions could also be utilized. Recall the cumulative distribution function of the exponential distribution as

$$F(y) = \lambda \cdot \exp(-\lambda y) \tag{5}$$

for $y \geq 0$. This results in the following calculation where we multiply the acceptance function (2) with the cumulative distribution function for the exponential distribution which gives us the probability that customer $b \in PC$ arrives in interval $(a_1, a_2]$ weighted with the resulting acceptance probability of the optimal insert, i.e. the expected acceptance probability:

$$\begin{aligned} &\int_{a_1-T}^{a_2-T} \underbrace{\exp\left(-\mu \cdot \frac{B_{b^-} - (T+y) - t_b}{t_b}\right)}_{(2)} \cdot \underbrace{\lambda \cdot \exp(-\lambda y)}_{(5)} \mathrm{d}y \\ &= \lambda \cdot \exp\left(-\mu \cdot \frac{B_{b^-} - T - t_b}{t_b}\right) \cdot \left[-\frac{t_b \cdot \exp\left(-\frac{(\lambda t_b - \mu)y}{t_b}\right)}{\lambda t_b - \mu}\right]_{a_1-T}^{a_2-T} \end{aligned} \tag{6}$$

Note that the request time e_b is replaced by $(T + y)$ in the acceptance function, since that is the arrival of the customer into the system if we are currently at the point in time T. The interval boundaries are adjusted by the current point in time T to determine the interarrival times correctly given that we are in T. Formula (6) holds for an interval $(a_1, a_2]$ only if the optimal insertion location (i.e. the one that maximizes the acceptance probability) is the same for all request times of the customer within the interval, i.e. the value B_{b^-} is constant. This is the case for each determined interval. For the last interval of each tour, the calculation changes slightly. Since the acceptance probability is then constant (i.e. independent of y), the term simplifies to

$$\int_{a_1-T}^{a_2-T} \exp\left(-\mu \cdot \frac{t_{\sigma_{\bar{n}}, b^+}}{t_b}\right) \cdot \lambda \cdot \exp(-\lambda y)\mathrm{d}y = \lambda \cdot \exp\left(-\mu \cdot \frac{t_{\sigma_{\bar{n}}, b^+}}{t_b}\right) \cdot \left[-\frac{\exp(-\lambda y)}{\lambda}\right]_{a_1-T}^{a_2-T}. \tag{7}$$

When evaluating the intervals across multiple tours for a single possible customer $b \in PC$, we need to look at their intersections. This is visualized in Fig. 4. The two tours have different intervals that were determined by Algorithm 1. At the bottom of the figure, the intersections are shown. For each intersection, we can then calculate the weighted acceptance probability by using the best acceptance probability across all tours, since we care only about the best possible offer we could make the customer $b \in PC$.

When determining the best tour for each intersection interval, we need to exclude certain tours, however. This is best illustrated by a small example: Let i be a new dynamic customer that arrives into the system and we now want to evaluate the effect on the potential if we were to insert the customer i into tour $k = 1$. In such a case, the tour $k = 1$ is "ruined" and should not be considered when calculating the potential, as we calculated the potential for tour k without the new customer i. However, after

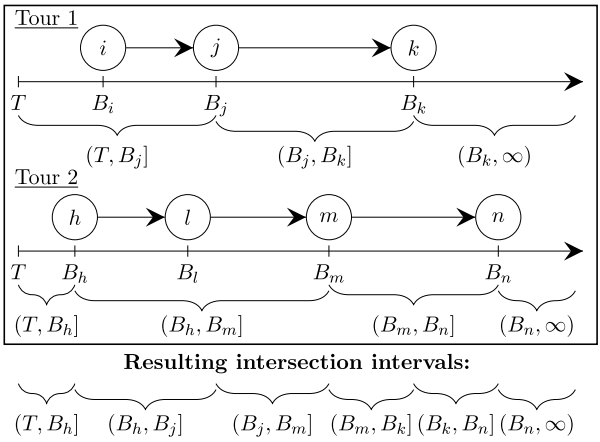


Fig. 4 Example for the intersection intervals

inserting customer *i* into tour *k* we need to recompute its potential, which is not done while inserting a customer due to computational effort (but later in the meantime before the next customer's arrival). We, therefore, assume that if a tour is changed by inserting a customer into it, we lose the potential it had for any possible customers (and is thus ruined). Additionally, if there are precomputed tours K'_1, which are tours that were originally based on tour $k = 1$, i.e. are tour 1 but with one removed customer, they are also ruined.

If we insert into a tour $k \in K'$, we need to consider the fact that another tour will be ruined by the reinsertion of the customer that was removed to create tour $k \in K'$. This entire procedure is shown in Algorithms 2 and 3. Algorithm 2 calculates the potential for a certain possible customer $b \in PC$ and a subset of tours $\bar{K}$, i.e. calculates $PT_{b\bar{K}}$. Algorithm 3 takes these values and averages them over all possible customers $b \in PC$. The set of ruined tours $\hat{K}$ in Algorithm 2 and the $\bar{K}$ from the returned value $PT_{b\bar{K}}$ are related, since we can use all not ruined tours for the calculation of the potential. Thus, $\bar{K} = (K \cup K')\backslash\hat{K}$.

1: **function** CALCULATEPOTENTIAL(tours $K \cup K'$, tours to exclude $\hat{K}$, possible customer $b \in PC$)
2: $PT_{b\bar{K}} \leftarrow 0$ Potential of tours $\bar{K} = (K \cup K')\backslash\hat{K}$
3: **for all** interval boundaries of customer b across the tours $\bar{K}$ **do**
4: $T_1 \leftarrow$ previous boundary of interval
5: $T_2 \leftarrow$ next boundary of interval
6: Find the best tour for the interval $(T_1, T_2]$, calculate its expected value by using formula (6) (or (7) if it is the last interval in a tour) and add that to $PT_{b\bar{K}}$
7: **end for**
8: **return** $PT_{b\bar{K}}$
9: **end function**

Algorithm 2 Calculation of potential

function CalculateTotalPotential(tour that is inserted into $k \in K \cup K'$, possible customers PC)
  $best \leftarrow 0$
  if $k \in K$ **then**
    $\hat{K} \leftarrow \{k\} \cup K'_k$ Set of ruined tours
    $best \leftarrow \frac{1}{|PC|} \sum_{b \in PC}$ CalculatePotential$(K \cup K', \hat{K}, b)$
  else if $k \in K'$ **then**
    $K'' \leftarrow$ all possible tours that we can insert the customer into who is removed from the original tour to get tour k
    for all $k'' \in K''$ **do**
      $\hat{K} \leftarrow \{k, k''\} \cup K'_k \cup K'_{k''}$ Set of ruined tours
      $best = \max(best, \frac{1}{|PC|} \sum_{b \in PC}$ CalculatePotential$(K \cup K', \hat{K}, b))$
    end for
  end if
  return $best$
end function

Algorithm 3 Calculation of total potential

5.2.3 Using the potential

Finally, if the potentials were all calculated, Algorithm 4 gives the evaluation of an insert of a dynamic customer including the potential. Like before, the acceptance probability *prob* is an important criterion. However, a second part of the objective is now the potential over all tours that are not ruined by the respective insert. The two parts are combined in a weighted sum where $0 \leq \omega \leq 1$ gives the weight of the potential. It should be noted that all potential related values are solely calculated in the idle phase between two arriving dynamic customers. Thus, unlike the pseudo code might suggest, the values calculated by Algorithms 1–3 are all saved and read from memory if needed and not calculated when the new dynamic customer's inserts are evaluated during the execution of Algorithm 4. Should the calculation of the potential be canceled early, the value returned by the CalculateTotalPotential function in Algorithm 3 will be zero.

$PC \leftarrow$ all potential customer requests
$j \leftarrow$ customer that just made a request and for whom we are evaluating inserts
for all $k \in K \cup K'$ **do**
  Use Algorithm 5 to find the optimal insert of customer j into tour k
  $prob \leftarrow$ acceptance probability of that insert
  $PT_{\text{total}} \leftarrow$ CalculateTotalPotential(k, PC)
  Weight the insert with the value $(1 - \omega) \cdot prob + \omega \cdot PT_{\text{total}}$
end for
Choose the insert with the highest value

Algorithm 4 Evaluation of inserts

We can also use the potential in another step of the algorithm. When we choose to insert a dynamic customer into a tour $k \in K'$, i.e. a precomputed insert where a customer *i* is removed and reinserted into a different tour, we need to decide where to reinsert customer *i*. If there is only a single tour we can feasibly insert customer *i*

into, it is chosen automatically. If there are multiple tours, we choose the tour that has the least impact on the potential.

6 Computational study

In the computational study, we evaluate the performance of our methods. Our main aims are to evaluate the trade-off between the current customer's interest and the future customers' interests as well as the effects of clustering, parallelization, and sampling. In the first subsection, we describe the instance generation, in the second the computational setup, and in the third the results.

6.1 Instance generation

Our instances use the virtual stops of a former ridepooling provider in a district of Hamburg, Germany. We assume that these stops have been defined a priori and are, therefore, also known to our algorithm (and can thus be used in the calculation of the potential in Sect. 5). There are 120 stops in total. Every stop is a potential pickup and/or delivery location. Pickup and delivery location of a request are determined uniformly distributed. The only constraint is that they are two different locations. The resulting average direct travel time for a request is roughly 8.5 min.

We draw the interarrival times between two consecutive requests from an exponential distribution with parameter λ. This is in line with other authors, e.g. Sheridan et al. (2013) and Tafreshian et al. (2021), who assume a Poisson process for the arrivals, which results in an exponential distribution for the interarrival times. The vehicle capacity Q is assumed to be 6, which is in line with practical applications. The number of passengers q_{i^+} of request i is drawn from an exponential distribution with $\lambda = 0.9$ and the resulting value is rounded up. Should it be larger than 6, it is discarded and redrawn (Pfeiffer and Schulz (2022b)).

6.2 Computational setup

We varied the parameter for the interarrival times $\lambda \in \{0.0111, 0.0167, 0.0333\}$ which results in average interarrival times $(1/\lambda)$ of 90, 60 and 30 s, respectively. The number of customers was set to $n \in \{100, 250, 500, 750, 1000\}$ and the number of vehicles to $k \in \{6, 8, 10\}$. For each combination of n, k, and λ we generated 20 instances (900 in total). The acceptance factor μ was set to 0.5 in our experiments. In preliminary tests, a factor of 0.5 was reasonable such that not too many customers reject while detours are not too long. A factor of 0.5 means that a relative detour of 100% is accepted with a probability of roughly 60%. Remember that we consider a setting in a large city with small direct travel times. Thus, the detour is still just a few minutes while it includes the driven detour as well as the waiting time before being picked up. The maximum allowed shift of a customer's time window l, i.e. the length of the delivery node's time window, is set to 900 s (15 min), which is also done this way in practice. We vary l in Sect. 6.3.5 to evaluate the effect of the time window length. The expected time horizon for each instance can be calculated by n/λ.

The decision whether a customer accepts a given offer is dependent on a stochastic variable. Thus, each instance is dependent on an initial seed for the random number generator. To increase the robustness of our results, we solve each instance three times with different random seeds to simulate different customer behaviours. These three seeds are fixed, so that we effectively end up with a test set of 2700 instances. In the following, the term instance refers to the specific instance-seed-combination.

We ran different tests to see whether precomputation and potential are effective and to see how much influence the maximum time shift l (Sect. 6.3.5), a clustering of virtual stops (Sect. 6.3.6), a sampling of only a subset of potential future customers in the potential calculation (Sect. 6.3.7), as well as parallelization (Sect. 6.3.8) have. In order to test the effectiveness of the precomputation and potential methods and also determine good parameter settings, we solved each of the 2700 instances with different configurations. In the first two configurations, we turned off the potential and varied whether we turn the precomputation off (C_{B0}) or on (C_{B1}). We can interpret C_{B0} as a greedy decision maker who always inserts the current request in the best possible way into the tours without reassigning another customer or having in mind future customers. In setting C_{B1}, the decision maker is smarter and includes reinserts of already accepted customers into the decision. These two configurations serve as benchmarks. For the other configurations (referred to as C_{ω_χ}), we had the precomputation and potential turned on and varied the potential weight factor ω as well as the number of clusters χ to consider for the potential: $\omega \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$ and $\chi \in \{10, 20, 30, 40, 60, 120\}$. The last setting for χ indicates no clustering since there are 120 stops. The decision maker in settings C_{ω_χ} can be interpreted as a smart one who includes the interests of future customers into the decision on the insert of the current customer. The higher ω is, the more value is given on future customers. Parameter χ can be interpreted as technical conditions to reduce the computational effort in this time critical setting. The smaller χ is, the more stops are clustered and the smaller are computation times but the more inaccurate are the results. We ran a full factorial design, i.e. including the initial two benchmark configurations we have 32 configurations, resulting in 86,400 observations in total for the first test. In the first test, everything was run on a single thread.

In a second test, we took the best configuration as identified in the first test and used it to solve the instances again but with varying l, a varying sampling size, and varying numbers of threads.

The computations were all run on a machine with an AMD Ryzen Threadripper 3990X with 2.9GHz and 256GB RAM available. The code was written in C++ and compiled with the MSVC compiler (version 1916). For the parallelization, we used the Microsoft Parallel Patterns Library. All used instances can be found under the following link: https://doi.org/10.25592/uhhfdm.14275.

6.3 Results

In the following, $\bar{R}$ refers to the average fraction of rejected customers. We first evaluate the parameter choices in Sect. 6.3.1.

6.3.1 Parameter evaluation

Figure 5 shows the average rejection rate for the best performing setting dependent on the number of vehicles $|K|$ and the interarrival time $1/\lambda$. The best configuration is the one that managed to achieve the lowest average fraction of rejected customers in its solutions. The respective best configuration is also indicated for each setting. Configuration $C_{0.3_30}$, for example, is the best configuration in the bottom left setting even though it did not do very well over the entire set (see Table 7). This is compounded by the fact that in the bottom left setting, the system is likely overflowing with customers: There are few vehicles and a small interarrival time. Thus, it is possible that the advantage of otherwise good configurations is diminished, as the vehicles are all full and rejection rates are high either way, no matter how smart the algorithm's decisions are. The average values follow the expectations for the other settings: If the number of vehicles or the interarrival time increases, the system is not as busy and can thus serve more requests. As a conclusion, our choice of interarrival times and number of vehicles lead to a set of sufficiently challenging instances for the evaluation of the presented methods.

Moreover, the figure shows that average rejection rates strongly depend on interarrival times and number of vehicles. This leads to our first insight:

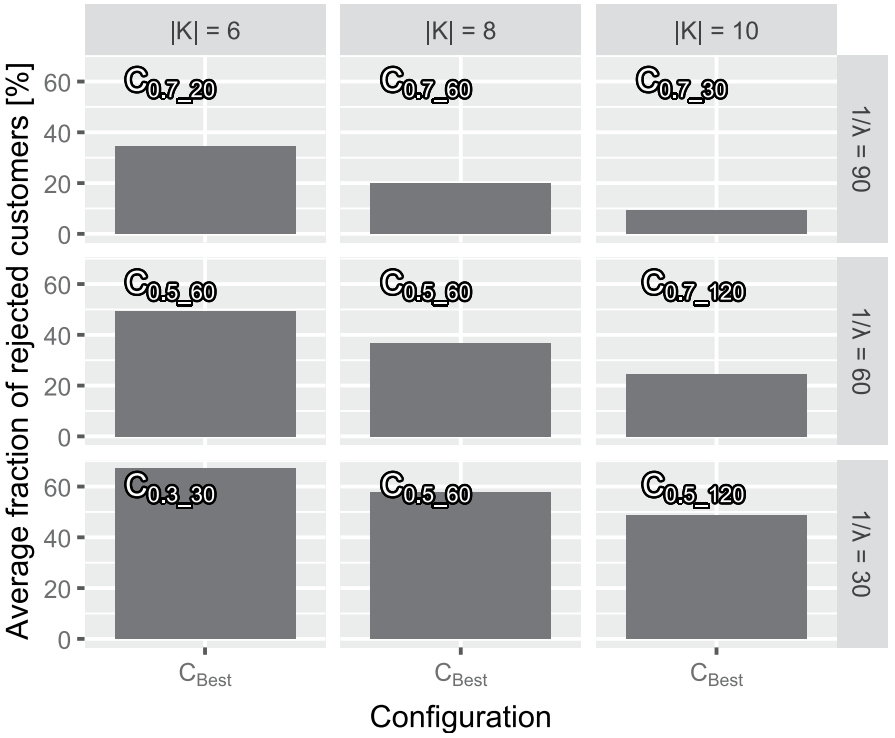


Fig. 5 Evaluation for different interarrival times $1/\lambda$ and number of vehicles $|K|$

Finding 1 *Depending on the arrival rate of the customers, a service provider can adjust the number of vehicles in service to steer the number of customers which are rejected on average and thereby the system's performance.*

This is a result which has also been observed in other studies (Pfeiffer and Schulz 2022a; Schulz and Pfeiffer 2024a). In the next section, we evaluate our new components precomputation and potential.

6.3.2 Evaluation of precomputation and potential

In Fig. 6, we evaluate the effect of precomputation and potential. It shows the average relative deviation of the two benchmark configurations and the best configuration in each setting from the best known solution aggregated over all instances and split according to number of vehicles $|K|$ and interarrival time $1/\lambda$. The best configuration for each setting is also shown (C_{Best}). The relative difference for each instance is calculated as $(R_y - R^*)/R^*$, where R_y is the fraction of rejected customers when using configuration C_y ($y \in \{B0, B1, Best\}$) and R^* is the minimum fraction of rejected customers for that instance found over all configurations. The best configurations each feature positive deviations, since no configuration was able to find the best solution in each instance of a setting.

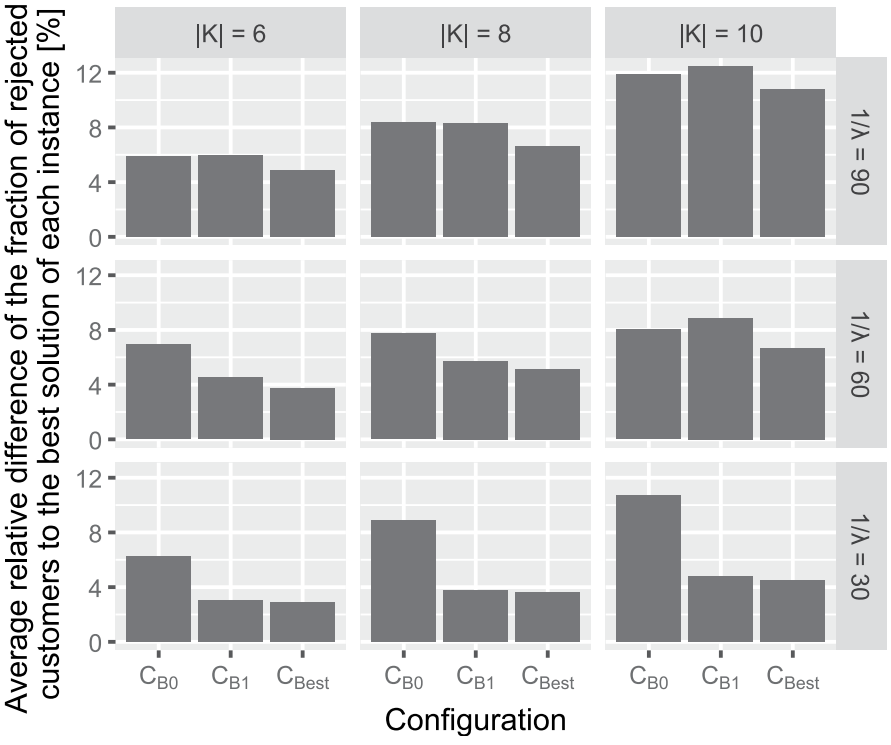


Fig. 6 Evaluation of precomputation and potential

In general, the difference between the two benchmark configurations can show the effect of the precomputation component. If C_{B1} is better, it means that the precomputation was responsible. The difference between C_{B1} and C_{Best} can show the effect of the potential component, as the only thing that differs between those two configurations is the use of potential. If C_{B1} performs poorly relative to C_{Best}, it means that the inclusion of the potential component led to a large improvement in the solution quality. This seems mostly to be the case in the upper six settings, i.e. with not too small interarrival times. The absolute values cannot be compared across the settings. The high average relative deviations in the upper right setting, for example, reflect a higher volatility in the results of the configurations: Even the best configuration was on average more than 10% away from the best known solution which is more than in the other settings.

Figure 6 shows that, when increasing $1/\lambda$ and decreasing $|K|$, the effect of the potential (difference between C_{B1} and C_{Best}) tends to decrease. This implies that the potential is most useful when there is actual slack for new customers to be inserted into. If there are few vehicles and/or customers arrive very quickly one after another (i.e. the utilization of the system increases), the tours likely do not offer many possibilities for new insertions. C_{B1} tends to perform better than C_{B0}. This underlines the general usefulness of the precomputation component though there are exceptions e.g. for 10 vehicles and a large $1/\lambda$, i.e. a large interarrival time, where C_{B0} is slightly better.

Our explanation for this result is the following: In the upper right setting, the system utilization is low, i.e. when a new customer arrives there is a high likelihood of being able to insert them into one of the tours in a reasonable way. Here precomputation is not needed. It could even be that constantly shifting customers around due to the precomputation hurts the solution quality in the long run in such a case: If we assume that a high acceptance probability for an insert correlates with the inserted customer being a good fit for the tour (e.g. is geographically close to other customer locations in the tour), then reassigning that customer later to another tour likely results in the customer not fitting in as well. A poor fitting customer will lead to higher detours which means in the long term the vehicles might be bounded for longer. While this effect can happen for any setting, in most situations the advantages of the precomputation (the current customer gets a better offer) seem to outweigh the possible downside.

This can be summarized in our second finding:

Finding 2 *Precomputation and potential are best used if the system is neither under- nor overutilized such that a good planning is necessary to serve additional customers, but there is still slack for new customers in the system.*

We continue in the next section with an evaluation of parameter ω, i.e. of the extent the potential should be included in the decision on the current customer's insert.

Table 1 The different ω values ordered according to the average fraction of rejected customers

ω	$\bar{R}$
0.5	38.83
0.7	38.87
0.3	38.89
0.1	38.92
0.9	40.69

Table 2 Difference of the three configurations on selected statistics. All values are in seconds

Name	TT_A	TT_R	TTM_A	TTM_R	Delay
C_{B0}	561.00	459.23	390.51	380.99	181.71
C_{B1}	563.43	454.15	390.83	380.19	279.87
$C_{0.5_120}$	563.21	454.29	390.70	380.42	273.20

6.3.3 Evaluation of the potential parameter

The aggregated results over all instances for different values of ω can be found in Table 1. In each of the rows, the average rejection rate over all 16,200 instances (2,700 instances for each of the six χ values with all configurations C_{ω_χ}) with the corresponding ω are presented. It holds for each row that the result is significantly better with $p < 0.001$ than the result in the next row. For the values of ω, it can be seen that the values close to the middle are better than the extremes. It appears to be better to weight the potential value roughly equally to the acceptance probability of the insert itself. Weighting the potential very highly ($\omega = 0.9$) causes the results to deteriorate massively. This means that in the trade-off between current and future customers a balanced approach seems to work best.

Finding 3 *The potential is used best the closer the weighing parameter ω, which determines the extent to which opportunity costs for future customers are included in the decision on the current customer's insert, is to 0.5. This means that both, interests of current and future customers, should be weighted equally.*

In the following sections, we restrict our analysis to configuration $C_{0.5_120}$, i.e. with $\omega = 0.5$ and $\chi = 120$ (no clustering; exception: Sect. 6.3.6 where clustering is evaluated). We refer to Appendix H for a detailed comparison of all configurations.

Using the potential can change the structure of accepted customers. We evaluate this aspect in the next section.

6.3.4 Influence of potential on served customers

In order to test whether the potential has an effect on the type of customers our system serves, we present a comparison between the two benchmark configurations as well as Configuration $C_{0.5_120}$. We want to know whether the direct travel time of a customer's request or the specific pickup and delivery locations (close to the center or on the outskirts) have an influence when considering the potential. Our findings are shown in Table 2. The table uses the following notation, whereat all values are given in seconds:

- TT_A: Average direct travel time of accepted customers.
- TT_R: Average direct travel time of rejected customers.
- TTM_A: Average travel time of accepted request locations to middle of service area.
- TTM_R: Average travel time of rejected request locations to middle of service area.
- Delay: How much an accepted customer's final delivery time is delayed compared to the original offered delivery time.

On average, the accepted customers feature longer direct travel times than the rejected ones. This is an interesting and somewhat unintuitive result, as it is easier for the system to serve short trips which occupy the vehicle for a shorter time. This holds especially because all customers are equally valuable for the objective function accepting as many trips as possible. A reason for the result seems to be the choice of our acceptance function (2) which depends on the relative detour (1). Given a certain absolute detour $B_{i^-} - e_{i^+} - t_i$, a longer direct travel time t_i leads to a smaller relative detour *RD* and thereby to a higher acceptance probability. This makes it more likely that customers with a longer direct travel time accept our offer. In the comparison of the three configurations, there are only very small differences between the configurations.

There is also no large effect of the customer's area on the acceptance or rejection. Thus, there is no bias towards accepting requests solely in the center of the service area.

Finally, the delay differs between the configurations. In Configuration C_{B0}, a customer has to wait roughly three minutes longer until reaching their delivery location compared to the original time they are told. If we enable the precomputation, this value increases by about 1:40 min on average; this is because the precomputation will use the degree of freedom to shift already accepted customers to make room for new customers. When also utilizing the potential ($C_{0.5_120}$), this effect is slightly reduced, as the interests of future customers are already included to some extent in the initial insert.

We can summarize the results in the following finding:

Finding 4 *Precomputation and potential do not change the structure of accepted requests, but they are able to use the additional degree of freedom to delay already accepted customers to some extent to offer new customers a good trip. Furthermore, the relative detour leads to the acceptance of rather longer trips.*

In the next section, we vary the degree of freedom to delay accepted customers.

6.3.5 Influence of maximum time shift

We also investigated how much the maximum time shift l matters for our results, i.e. how large the time window for the delivery locations is. We varied the time shift between 0, 5, 10, 15, 20, 25, and 30 min (with 15 being the value we used for all other tests). The considered configurations are C_{B0}, C_{B1} and $C_{0.5_120}$. The average

results for varying l are shown in Fig. 7. Expectedly, a larger time shift lowers the rejection rates, as the time windows are larger, i.e. the degree of freedom to delay accepted customers in favour of new customers is higher. However, the effect quickly diminishes as the time shift increases. While the difference between $l = 0$ and $l = 5$ is fairly large, the difference between $l = 10$ and $l = 15$ is already much smaller. This is helpful, since under our assumptions, the time shift l is communicated to the customer beforehand. Thus, being able to choose a smaller value without losing much solution quality is crucial to attract customers to the service. Note that even for a maximum time shift of 30 min the average fraction of rejected customers does not decrease below 0.38. The results are in line with those of Sect. 6.3.4 where we have seen in Table 2 that the average delay is smaller than 5 min. Figure 7 specifies the results by adding that there can be a substantial variance in the delay of single customers. Nevertheless, together with the small average, this also shows that most of the customers have a short delay.

It follows Finding 5:

Finding 5 *The allowed maximum time shift l is an important aspect of the system. A small maximum time shift leads to significantly higher rejection rates while a larger maximum time shift makes the system unattractive for accepted customers. A reasonable trade-off is to set the maximum time shift to a value of 10 to 15 min.*

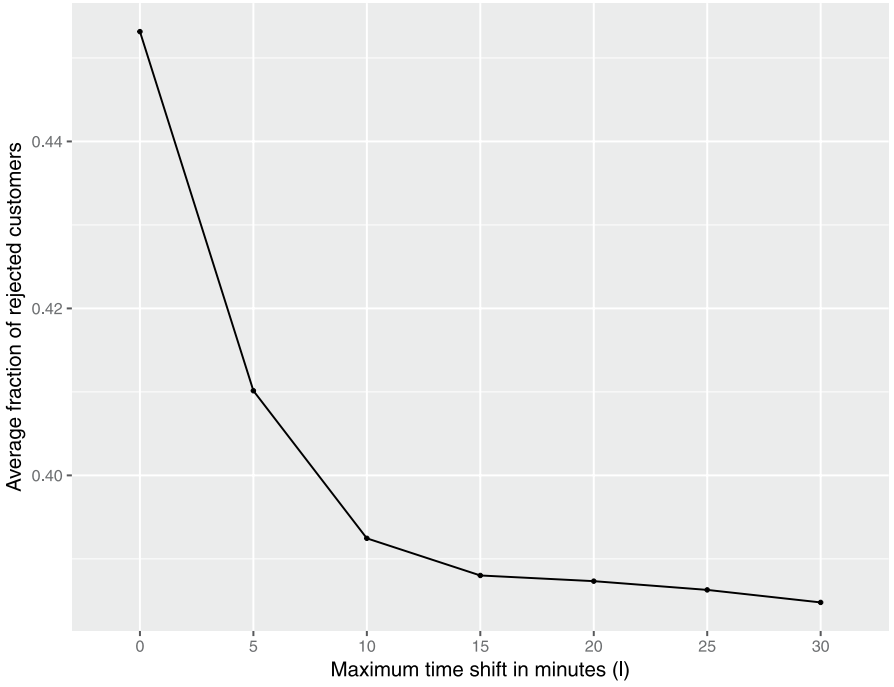


Fig. 7 The effect of the maximum time shift l on the average fraction of rejected customers

In the following sections, we evaluate methods to reduce the computational effort for our method, i.e. answer the question how it can be transferred to an application with a high number of virtual stops. We start with a clustering of the virtual stops (Sect. 6.3.6) before we introduce a sampling in the potential calculation (Sect. 6.3.7) and evaluate a parallelization method (Sect. 6.3.8).

6.3.6 Evaluation of clustering

Analogously to Sect. 6.3.3, Table 3 presents the aggregated results over all instances for the different values of χ. Each row represents the average of 13,500 instances (2,700 instances for each of the five ω values with all configurations C_{ω_χ}). It holds for each row that the result is significantly better with $p < 0.001$ than the result in the next row. There are only two exceptions: First, $\chi = 120$ which leads to a significantly smaller rejection rate than $\chi = 60$ with $p < 0.01$ and, second, $\chi = 40$ which does not lead to a significantly better result than $\chi = 30$ but than $\chi = 20$ ($p < 0.001$). The last column presents the computation times for the corresponding setting. They include the entire average time to solve the instances. Thereby, we jump to the next arrival time if all calculations are terminated before the next customer arrived, i.e. the not used time is not included in the computation times. If the next customer arrives before all calculations are terminated, we interrupt the current calculations and directly proceed with the new customer.

For the cluster sizes χ, the results are as expected: If there are more clusters (i.e. fewer locations are aggregated), more information is given to the potential calculation and the results are better. However, the differences in the results between the cluster levels are very small while computation times can clearly be reduced. This is beneficial for applications with a larger set of underlying virtual stops where clustering is likely necessary to keep running times short such that they are terminated before the next request occurs.

Finding 6 *Clusters can be used to reduce computational effort substantially while losing only a small amount of performance. The more clusters are used (the fewer stops are clustered together) the better is the performance, but the higher are computation times.*

Table 3 The different χ values ordered according to the average fraction of rejected customers

χ	$\bar{R}$	Time [s]
120	39.16	234.53
60	39.19	63.84
40	39.22	40.93
30	39.22	35.26
20	39.26	32.57
10	39.38	32.04

Table 4 The results for the different sampling sizes S

S	$\bar{R}$	Time (s)
100	38.95	32.05
500	38.93	33.73
1000	38.92	39.74
14,280	38.81	231.72

Table 5 The development of the computational time (wall-clock) when increasing the number of threads η

η	Time [s]
1	231.72
2	170.48
4	134.71
8	113.22
16	105.78

6.3.7 Sampling requests in potential calculation

In an additional test, we evaluated whether a sampling approach for the possible future customers could work to reduce the computational load for the potential calculation. The sampling works as follows: In every iteration, a random subset of the possible customers with size S is selected and only those possible customers are considered for this iteration. Since we sample a new subset in every iteration, this approach should average out any outliers. We investigated four different sampling sizes $S \in \{100, 500, 1000, 14{,}280\}$ where $S = 14{,}280$ are the previous results of Configuration $C_{0.5_120}$ including all possible customers ($14{,}280 = 120 \cdot 119$, i.e. customers starting from any virtual stop and going to any other). The calculations for the other sampling sizes also use this configuration.

Table 4 shows the sampling results in the same way as Table 3. As can be seen, the results are generally fairly close meaning that sampling leads only to a small performance reduction. Even if we only sample 100 possible future customers the solution quality decreases only by 0.14%. On the other side, computation times can be reduced substantially by the sampling approach. This means that sampling can be used to apply the algorithm for system's of larger size.

Finding 7 summarizes the results:

Finding 7 *Sampling can be used to reduce the computational effort substantially by only losing a small amount of performance. The higher the sampling size, the better is the performance, but the higher are the computation times.*

6.3.8 Parallelization

The previous results were all calculated with a single thread, i.e. sequentially. To investigate the effect of parallelization, we solved each instance with Configuration $C_{0.5_120}$ and varied the number of threads η. The average time over all instances

according to each setting of η is shown in Table 5. As for clustering and sampling the time is the entire time used for solving the instance. Thereby, we jump to the next customer arrival whenever all precalculations are finished. Expectedly, the real time when running the algorithm decreases as more threads are used, with the first doubling of threads reducing the time by roughly 25%. This means that parallelization can be used to apply the algorithm for system's of larger size. However, since the times are aggregated over all instances, the heterogeneity of the instances is not considered. In instances where the potential calculation takes longer, parallelization can realize more time savings. This is visualized in Fig. 8 where the wall-clock time is shown when using different multiple threads compared to running the algorithm on a single thread.

Finding 8 *Parallelization is beneficial to reduce the running time of the algorithm's components. By it, we can terminate the precalculations done in the meantime between two customer arrivals earlier and, thereby, increase the chance that all calculations can be done before the next customer arrives.*

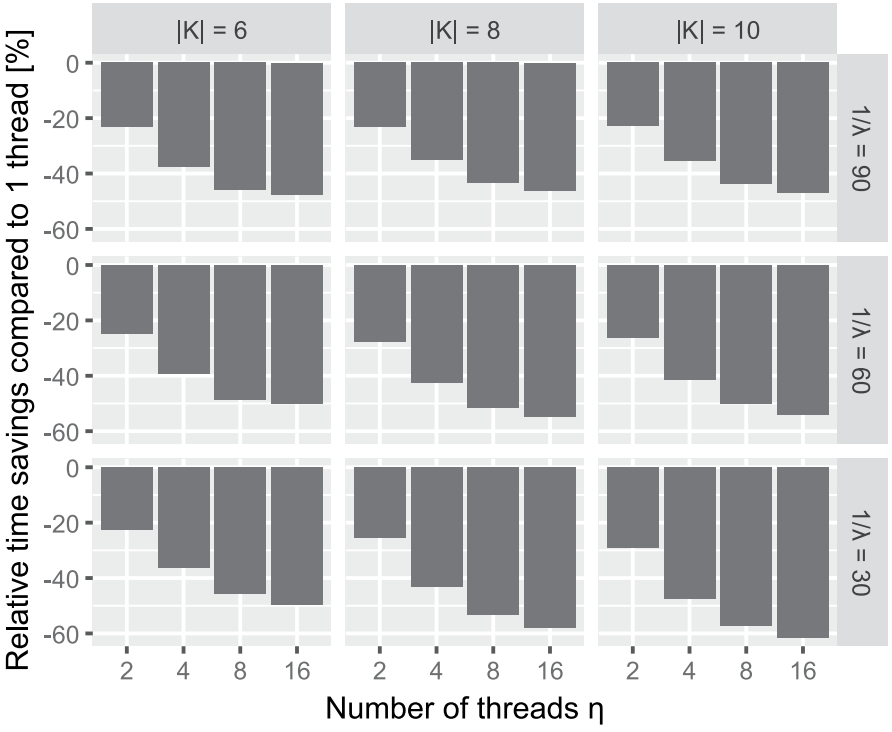


Fig. 8 Average time savings when using multiple threads instead of running the algorithm sequentially

7 Conclusion

In this paper, we present a novel stochastic-dynamic DARP which is oriented at the practical setting of a ridepooling provider in a large city. As such a provider is in competition with several other modes of transportation, the provider needs to offer tours with short waiting and travel times for its customers to maximize the acceptance rate. We use the relative detour to compute the customer's acceptance probability. Moreover, we solve the trade-off between the current customer's interest and future customers' interests by maximizing a weighted objective function with the acceptance probability of the current customer and the potential of the tours for an insert for future customers. We found that both should be weighted equally to maximize the acceptance rate and that the system's performance strongly depends on the time window size at delivery locations which determine the degree of freedom the system has to reinsert accepted customers in favour of new customers. A too small time window size leads to high rejection rates while a high one is unattractive for customers. Our results show that a time window length of 10 to 15 min is reasonable.

Moreover, we introduced several methods to improve the computational performance including precomputation in the meantime between two customer requests, sampling, clustering, and parallelization. We found that precomputation and potential change the structure of accepted requests only marginally while the acceptance probability depending on the relative detour leads to the acceptance of rather longer trips. Sampling, clustering, and parallelization allow to solve instances of reasonable size in a short time and can, therefore, be used to apply the methods in a setting with a large number of virtual stops. Furthermore, our method allows the provider (i.e. with historical data) to steer the number of used vehicles such that a given service level, i.e. acceptance rate, is reached.

In future research, our method can be adapted to include different customer priorities which can be addressed by varying the weight of a possible future customer in the calculation of the potential as well as the weights in the objective function when inserting them. Additionally, the influence of wait and ride time of a customer on the acceptance function can be weighted differently according to the customer's preferences. Besides, the length of the delivery time windows l can be adapted to differentiate between customers, e.g. according to the direct travel time (our results show that rather longer trips are accepted if all customers have the same time window length), and allow for price differentiation between customers or to offer customers shorter travel times for a higher price as well as normal travel times for a cheaper price. Moreover, different probabilities for the occurrence of certain requests can be evaluated.

Table 6 Overview of notation

B_j	Arrival time at location *j*
C_y	Configuration with characteristics *y*
D	Set of delivery locations
D_i^-	Last location before i^- in a tour in state S_i
e_{i+}	Request time of request *i*
γ_c	Number of stops in cluster *c*
i, b	Request/customer
i^+	Pickup location of customer *i*
i^-	Delivery location of customer *i*
I	Set of virtual stops
k	Index for vehicles
K	Set of tours
K'	Set of tours without a removed customer
$\hat{K}$	Set of ruined tours
$\|K\|$	Number of vehicles
l	Allowed shift of arrival time at delivery location
l_{i-}	End of time window at delivery location of customer *i*
λ	Parameter of exponential function with mean $1/\lambda$
μ	Weight parameter for relative detour
n	Number of requests
p	Position in a tour
p_k^i	Geographical position of vehicle *k* in state S_i
P	Set of pickup locations
P_i^-	Last location before i^+ in a tour in state S_i
PC	Set of possible customers
prob	Acceptance probability
$PT_{b\bar{K}}$	Potential of tours in set $\bar{K}$ for customer *b*
π	Policy
q^i	Vector with number of customers in the so far known requests
q_{i+}	Number of persons in request *i*
Q	Vehicle capacity
$\bar{R}$	(Average) rejection rate
RD	Relative detour
s_k^i	Tour schedule of vehicle *k* in state S_i
S_i	State after request *i* was realized (and scheduled)
σ	Sequence of locations in a tour
t_i	$= t_{i+,i-}$ direct travel time between pickup and delivery location of customer *i*
$t_{jj'}$	Travel time between locations *j* and j'
T	Time
V	Number of virtual stops
W_i	Characteristics of request *i* (i^+, i^-, e_{i+})
$\overline{W}_i$	Acceptance decision of customer *i*
ω	Weight of the potential
$x_i^{(\pi)}$	Offer for customer *i* (according to policy π)
χ	Number of clusters

Appendix A: Notation

The following table presents the used notation. For simplicity we omit notation here which is only used directly after its definition (Table 6).

Appendix B: Forward time slack

Gschwind and Drexl (2019) introduced a constant-time feasibility check based on the forward time slack (FTS) introduced by Savelsbergh (1992) and the observation that it must always be optimal to serve each location as early as possible. Let $\bar{n}$ be the number of locations in the tour and $p, p' = 1, 2, \ldots, \bar{n}$ the position in the tour. The FTS of p is defined as follows

$$F_p := \min_{p < p' \leq \bar{n}} \{w_{p,p'} + (l_{\sigma_{p'}} - B_{\sigma_{p'}})\},$$

where σ_p is the location which is visited at position p in the considered tour and $w_{p,p'}$ is the cumulative waiting time between positions p and p'.

Note that we consider a dynamic environment with immediate requests only, which means that requests are not known upfront. Thus, a vehicle can never be at the pickup location of request i and wait for the customer. Delivery locations also have no effectual restriction for the earliest arrival time—beside $e_{i^+} + t_i$, which is always fulfilled. Therefore, vehicles never wait in an optimal solution in our problem setting and the waiting time $w_{p,p'} = 0$ for all p and p'. However, we introduce the general version of the problem, which can also be used if customers can request a ride for a later point in time.

We set $l_{j^+} = l_{j^-} - t_j$ for all $j^+ \in P$. Thus, F_p is the minimum over all later visited locations j^+ of the cumulative waiting time between the location at position p and location j^+ and the time slack at location j^+ according to its time window. This means that an insertion after the location at position p must lead to a detour of at the most F_p time units. Otherwise, it is not feasible according to a time window of a later location. The insertion of a pickup location m^+ after the location at position p is feasible according to time windows if

$$t_{\sigma_p, m^+} + t_{m^+, \sigma_{p+1}} \leq F_p + t_{\sigma_p, \sigma_{p+1}}. \tag{8}$$

However, we have to insert the delivery location m^- as well at a later point of the tour after a position p'. Therefore, $F_{p'}$ has to be adapted by the resulting delay of the first insertion $\delta = t_{\sigma_p, m^+} + t_{m^+, \sigma_{p+1}} - t_{\sigma_p, \sigma_{p+1}}$ minus the cumulative waiting time of all locations between $p+1$ and p', i.e. $w_{p+1,p'}$. Thus, the delivery location's insertion is feasible according to time windows if

$$t_{\sigma_{p'}, m^-} + t_{m^-, \sigma_{p'+1}} \leq F_{p'} + t_{\sigma_{p'}, \sigma_{p'+1}} - \max\{\delta - w_{p+1,p'}, 0\}. \tag{9}$$

Certainly, the computations in (8) and (9) can be done in constant time given the tour schedule F_p for all positions p of the tour, and $w_{p,p'}$ for all pairs of positions p and p' of the tour with $p \leq p'$. Gschwind and Drexl (2019) argue that these numbers can be computed in quadratic run-time in $\bar{n}$ in the preprocessing. While the tour schedule can be computed in linear time because it is always optimal to visit a location as early as possible, we can omit to compute cumulative waiting times $w_{p,p'}$ between each pair of positions p, p' with $p \leq p'$. Instead, we introduce cumulative waiting times $\bar{w}_p$ up to the location at position p, which can be computed in linear time by setting

$$\bar{w}_p = \bar{w}_{p-1} + \max\left\{0, e_{\sigma_p} - \left(B_{\sigma_{p-1}} + t_{\sigma_{p-1},\sigma_p}\right)\right\}.$$

Because of $w_{p,p'} = \bar{w}_{p'} - \bar{w}_p$,

$$\begin{aligned} F_p &= \min_{p<p'\leq\bar{n}} \left\{\bar{w}_{p'} - \bar{w}_p + (l_{\sigma_{p'}} - B_{\sigma_{p'}})\right\} \\ &= \min\left\{\bar{w}_{p+1} - \bar{w}_p + (l_{\sigma_{p+1}} - B_{\sigma_{p+1}}), \min_{p+1<p'\leq\bar{n}} \left\{\bar{w}_{p'} - \bar{w}_p + (l_{\sigma_{p'}} - B_{\sigma_{p'}})\right\}\right\} \\ &= \min\left\{\bar{w}_{p+1} - \bar{w}_p + (l_{\sigma_{p+1}} - B_{\sigma_{p+1}}), F_{p+1}\right\} \end{aligned}$$

holds. Given $F_{\bar{n}} = l_{\sigma_{\bar{n}}} - B_{\sigma_{\bar{n}}}$, all FTSs can be computed recursively in linear time. Thus, the tour schedule, F_p for all positions p as well as the cumulative waiting times $\bar{w}_p$ for all positions p can be computed in linear time in $\bar{n}$. In total, feasibility of an insertion can be checked in constant time with linear effort in a preprocessing step. As our preprocessing steps are done during the stochastic time between two consecutive dynamic request times, the improved preprocessing run time is important for our application.

Appendix C: Detailed insertion procedure

Let m^+ be the pickup location and m^- be the delivery location of the new arriving customer. Let Q_p be the number of passengers in the vehicle after leaving the location at position p of the tour.

We consider a dynamic insertion in Algorithm 5. The first possible insertion of pickup location m^+ is at time $T = e_{m^+}$. Line 1 determines the next stop after T. Note that we do not change the next stop of a vehicle after it already started driving towards it. Line 2 restricts the for-loops starting in Lines 4 and 16 if a feasible insertion is already known. In 3, scalar g is the insertion point with the shortest detour (compare Lines 14–15), which is initialized with a sufficiently large number. Note that given a feasible insertion of pickup and delivery location after positions p and p', respectively, it is also feasible to insert the pickup location after any position p'' with $p < p'' < p'$ and $t_{\sigma_{p''},m^+} + t_{m^+,\sigma_{p''+1}} - t_{\sigma_{p''},\sigma_{p''+1}} < t_{\sigma_p,m^+} + t_{m^+,\sigma_{p+1}} - t_{\sigma_p,\sigma_{p+1}}$.

Given a tour with the sequence $\sigma_1, \sigma_2, \ldots, \sigma_{\bar{n}}$ and a current point in time T

1: Let $h = \text{argmin}_{p=1,\ldots,\bar{n}}\{B_{\sigma_p} \geq T\}$ and $h = \bar{n}$ if no $p = 1, \ldots, \bar{n}$ with $B_{\sigma_p} \geq T$ exists. In the latter case set $B_{\sigma_{\bar{n}}} := T$

2: Let $\tilde{n} = \text{argmax}_{p=1,\ldots,\bar{n}}\{B_{\sigma_p} + t_{\sigma_p,m^-} \leq o\}$ where o is the earliest arrival time of all feasible insertions of request m at delivery location m^-. If no feasible insertion exists so far, set $\tilde{n} = \bar{n}$ and $o = B_{\sigma_{\bar{n}}} + t_{\sigma_{\bar{n}},m^+} + t_m$.

3: Set $g = \max_{p=h,\ldots,\tilde{n}-1}\{t_{\sigma_p,m^+} + t_{m^+,\sigma_{p+1}} - t_{\sigma_p,\sigma_{p+1}}\} + 1$

4: **for** p = h to $\tilde{n} - 1$ **do**

5: **if** $t_{\sigma_p,m^+} + t_{m^+,\sigma_{p+1}} \leq F_p + t_{\sigma_p,\sigma_{p+1}}$ and $Q_p + q_{m^+} \leq Q$ **then**

6: Insert pickup location m^+ between locations σ_p and σ_{p+1}

7: **if** $t_{\sigma_p,m^+} + t_m + t_{m^-,\sigma_{p+1}} \leq F_p + t_{\sigma_p,\sigma_{p+1}}$ **then**

8: Insert delivery location m^- between locations m^+ and σ_{p+1}

9: **if** $B_{\sigma_p} + t_{\sigma_p,m^+} + t_m < o$ **then**

10: $o = B_{\sigma_p} + t_{\sigma_p,m^+} + t_m$

11: **end if**

12: Set $p = \tilde{n}$ and go to Line 29

13: **end if**

14: **if** $t_{\sigma_p,m^+} + t_{m^+,\sigma_{p+1}} - t_{\sigma_p,\sigma_{p+1}} < g$ **then**

15: $g = t_{\sigma_p,m^+} + t_{m^+,\sigma_{p+1}} - t_{\sigma_p,\sigma_{p+1}}$

16: **for** $p' = p + 1$ to $\tilde{n}$ **do**

17: **if** $Q_{p'} + q_{m^+} > Q$ **then**

18: Go to Line 29

19: **end if**

20: **if** $t_{\sigma_{p'},m^-} + t_{m^-,\sigma_{p'+1}} \leq F_{p'} + t_{\sigma_{p'},\sigma_{p'+1}} - \max\{\delta - (\bar{w}_{p'} - \bar{w}_{p'+1}), 0\}$ **then**

21: Insert delivery location m^- between locations $\sigma_{p'}$ and $\sigma_{p'+1}$

22: **if** $B_{\sigma_{p'}} + t_{\sigma_{p'},m^-} < o$ **then**

23: $o = B_{\sigma_{p'}} + t_{\sigma_{p'},m^-}$

24: **end if**

25: Set $\tilde{n} = p' + 1$ and go to Line 29

26: **end if**

27: **end for**

28: **end if**

29: **end if**

30: **end for**

Algorithm 5 Insertion

The core of the algorithm is a for-loop over all possible pickup insertions (Lines 4–30). If an insertion is feasible (Line 5), the pickup location is inserted (Line 6). If it is furthermore possible to insert the delivery location directly after the pickup location (Line 7), this is done (Line 8). As an insertion of the delivery location at any later point of the tour can never lead to a better solution (triangle inequality), the algorithm jumps to Line 29 and terminates because in all not considered cases even the pickup location would be inserted after position $p + 1$, but the delivery location is currently inserted before location $p + 1$ such that this can never be better (triangle inequality; Line 12). However, if the current solution is a new best one, it is saved (Lines 9–11).

The second part of the algorithm (Lines 14–28) inserts the delivery location. Line 14 ensures that delivery insertions are only evaluated if a new best solution is possible. This is only the case if the detour which results from an insert of m^+ between locations p and $p + 1$ is shorter than for the best known pickup insertion so far. Otherwise, the prior insertion of m^+ with a shorter detour results in no shorter FTS for all later locations and thereby not to a worse insertion of m^-. In Lines 16–27, all possible delivery insertions are evaluated, which works analogously as for the pickup insertions (Lines 5–13). However, feasibility according to the vehicle capacity Q

must be ensured at every location. Thus, the for-loop in Lines 16–27 is left if the capacity requirement is not fulfilled (Lines 17–19). The algorithm returns the earliest feasible arrival time *o* of customer m^+ at its delivery location.

Appendix D: Parallelization for insertion calculation and precomputation

The described algorithmic steps can be parallelized to improve the response times of the algorithm. The parallelization is described in the following. When a new dynamic customer arrives and needs to be inserted, the different tours into which the customer can be inserted are independent of each other. There are always at least $|K|$ tours available for insertion, but there might be more if the precomputation generated additional tours where customers have been removed (set K'). Each of these tours $k \in K \cup K'$ can be given to a single thread which then finds the best possible insertion point of that customer into tour *k*.

Since we are looking for the insert that maximizes the acceptance probability of the new customer, the threads communicate their best found arrival time *o* with each other. This allows for the skipping of certain inserts in Algorithm 5: Since the delivery positions are checked in order, if for pickup tour position *p* the next checked delivery tour position p' features an arrival time that is not better than the current global best, the remaining delivery tour positions $p'' > p'$ do not need to be checked for that pickup tour position *p*, as they can only worsen the arrival time and the pickup tour position can be incremented. This can be used even when the potential described in Sect. 5 is included.

The precomputation phase can be parallelized by considering each customer separately. For each customer, the inserts into the other tours are independent of each other. Naturally, two inserts into the same tour might conflict, but at most one of these inserts will be chosen to make room for the next customer.

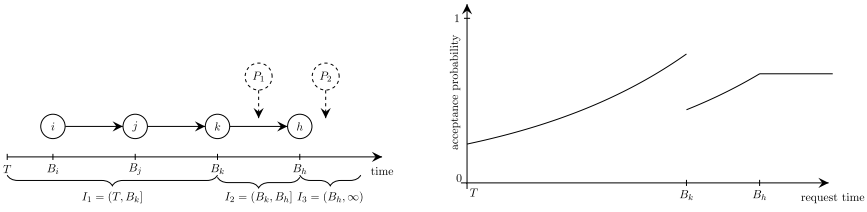
Appendix E: Example for determination of intervals

An example for the functionality of Algorithm 1 is shown in Fig. 9. In Fig. 9a, we are currently in point in time *T* and the tour contains four customer locations *i*, *j*, *k*, *h* (the vehicle is currently en route to *i*). For the new possible customer *b* we now try to find the optimal insert. For clarity we only depict the optimal insertion positions for the pickup node of the new customer P_l where *l* is the respective interval. W.l.o.g. the optimal insertion for the delivery node can be assumed to directly follow the pickup location. First, we call Algorithm 5 with the current time *T*. Let us assume it returns the optimal insert for the pickup node P_1. Since P_1 is inserted after location *k*, it means that this insert is optimal even if the customer enters the system after *T* (due to the triangle inequality and since we want an insert as early as possible). The insert stops being the optimal insert once we have visited location *k*, since we are then en route from *k* to *h* and cannot insert P_1 anymore. The first interval is thus $I_1 = (T, B_k]$. Then, we would call Algorithm 5 a second time with

B_k as the value for the current time. This returns us the optimal pickup insertion P_2 which is located after location h and results in the second interval $I_2 = (B_k, B_h]$. The final interval follows as $I_3 = (B_h, \infty)$. Both I_2 and I_3 have the same insertion position (after location h), but there is a difference: If the customer arrives into the system in interval I_2, the pickup and delivery nodes of the customer are visited at the same time ($B_{b^+} = B_h + t_{hb^+}$ and $B_{b^-} = B_{b^+} + t_b$, respectively) regardless of the actual request time as long as it is in the interval. However, if the customer places the request in interval I_3, the arrival time at the delivery node of the customer depends on the customer's request time, since we cannot travel to the customer before we know the customer has placed their request. This means the customer's detour is a constant given by t_{hb^+}. The acceptance probabilities are also visualized in Fig. 9b. In every interval but the last, the acceptance probability is given by an exponential function. From one interval to the next, this function can change due to the difference in arrival times from the variation in optimal insertion positions. Otherwise, within an interval the acceptance probability steadily increases the later the customer places their request, since the time between placing the request and arrival at the delivery location (and thus the detour) decreases. For the last interval, the acceptance probability is constant (at value $\exp\left(-\mu \cdot \frac{t_{hb^+}}{t_b}\right)$) due to the constant detour.

Appendix F: Parallelization for potential calculation

Like the algorithmic steps in Sect. 4, we also parallelize the calculation of the potential. Algorithms 1, 2, and 3 can all be run in parallel each. They still need to be calculated in order, e.g. the intervals for every tour $k \in K \cup K'$ in Algorithm 1 need to be determined before the best tour for each interval intersection is determined in Algorithm 2. If the number of stops to consider is very large, the size of the set PC becomes very large as well (if no clustering of stops is included). This increases the computational effort significantly for the proposed algorithms, as the potential is calculated for every possible customer $b \in PC$. The parallelization can alleviate the resulting issues on the overall performance by spreading the work over multiple threads.



(a) Tour with different insertion positions based on the new customer's request time and the resulting intervals.

(b) Acceptance probability functions for the new customer based on the above tour's insertion positions.

Fig. 9 Example for Algorithm 1

As mentioned in Appendix D, during the parallel execution of the for loop in Algorithm 4, the threads can still communicate which allows them to skip certain insertions if it is already clear that they cannot be better than the current best found insert.

We also implemented a dynamic batching procedure so that the dispatching of tasks to the threads does not become a bottleneck: Depending on the lengths of the tasks, a single thread might receive more than one task and will only ask for more tasks when the entire batch is done.

Appendix G: Clustering for potential calculation

To keep computation times small and also to deal with large sets of stops, we implemented a clustering procedure for the stops. Our distances are based on the actual city layout, i.e. are not simply euclidean. Thus, we cannot use the standard *k*-means algorithm to determine the centroids, since the resulting centroids might not be reachable by car (e.g. the centers of the clusters could be in the middle of an apartment complex or park). Therefore, we attempt to find medoids which guarantee that the representative of each cluster is one of the stops in the cluster. To solve this *k*-medoids clustering problem, we use the Partitioning Around Medoids Algorithm (Kaufman and Rousseeuw 1990, Chapter 2).

The clustering algorithm takes as input the number of clusters and is run once at the start of our algorithm. Once the clustering algorithm is finished, we treat each cluster, represented by its medoid, as a stop. Thus, if there are χ clusters, we have $\chi(\chi - 1)$ possible customers. When determining the probability of a certain possible customer to appear, we then also have to take into account the number of stops in each cluster, since a request from one large cluster to another large cluster is more likely to occur than a request between small ones. If we treat each cluster as a node in a graph, each arc between two clusters represents a possible customer trip and the arc is weighted with the number of stops in the pickup cluster and in the delivery cluster. Let γ_c be the number of stops in cluster $c = 1, \ldots, \chi$, then the weight for a customer $b \in PC$ who goes from cluster c to c' is given as

$$\frac{\gamma_c + \gamma_{c'}}{2(\chi - 1) \sum_{c''=1}^{\chi} \gamma_{c''}},$$

where the denominator is the sum of the arc weights so that we receive weights for each possible customer in the interval (0, 1). These weights are used instead of the default $\frac{1}{|PC|}$ weight in Lines 5 and 10 of Algorithm 3. Should more information about the probabilities of pickup-delivery pairs be available, it can easily be included in the weighting step and certain possible customers can be weighted higher or lower accordingly.

Appendix H: Evaluation of all configurations

An overview for all considered configurations is given in Table 7. The PreC and Pot columns indicate whether the precomputation and potential were used in the configuration, respectively. The asterisks in the $\bar{R}$ column indicate the result of a paired t-test with the benchmark configuration that had the potential disabled but the precomputation enabled (C_{B1}), i.e. shows the relative advantage of using the potential (*: $p < 0.05$, **:$p < 0.01$, ***: $p < 0.001$). The results for the configurations roughly follow the aggregated results for ω and χ we already presented in Tables 1 and 3: The best configurations utilize ω values of 0.5 and 0.7. The values for χ seem to differ more and are only of secondary relevance behind ω. It should be noted that there is still some noise remaining and between the three best configurations, there are no statistically significant differences in the $\bar{R}$ ($p < 0.05$). As seen in the table, each configuration was also tested against benchmark configuration C_{B1}. We only test against C_{B1} because it differs in exactly one component from all the other configurations: The other benchmark configuration (C_{B0}) does not use precomputation while all other configurations use the potential. Differences in the performance can therefore be attributed to the inclusion or removal of the respective component.

The best configurations are all significantly better than the benchmark configuration C_{B1}. This means that, if well configured, the inclusion of the potential calculation can significantly improve the solution quality. However, the parametrization plays a key role. If the potential is weighted too low ($\omega \in \{0.1, 0.3\}$) or does not receive fine enough information ($\chi \in \{10\}$), the potential does not offer much of an advantage. In the most extreme case with $\omega = 0.9$, the results are significantly worse. These configurations are also the only ones that achieve a worse average solution quality than the configuration without the precomputation enabled (C_{B0}). It seems that overvaluing the potential compared to the current customer's acceptance probability does not pay off, as the possible future gains of highly weighting the potential do not compensate for the worse acceptance probabilities.

Since Configuration C_{B0} (no precomputation) is significantly worse than Configuration C_{B1} (with precomputation), we conclude that precomputation is a worthwhile step to improve solution quality. Thus, overall both, precomputation as well as potential, can lead to a significant reduction in the average number of rejected customers.

Acknowledgments This research has been funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) – 530137180.

Funding Open Access funding enabled and organized by Projekt DEAL.

Declarations

Conflict of Interest The authors declare that they have no conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative

Table 7 Every considered configuration in ascending order of their average fraction of rejected customers $\bar{R}$

Name	PreC	Pot	ω	χ	$\bar{R}$ [%]	Name	PreC	Pot	ω	χ	$\bar{R}$ [%]
$C_{0.5_60}$	•	•	0.5	60	38.78***	$C_{0.3_60}$	•	•	0.3	60	38.9
$C_{0.5_120}$	•	•	0.5	120	38.81***	$C_{0.1_30}$	•	•	0.1	30	38.9
$C_{0.5_20}$	•	•	0.5	20	38.83**	$C_{0.1_120}$	•	•	0.1	120	38.9
$C_{0.5_40}$	•	•	0.5	40	38.84**	$C_{0.1_60}$	•	•	0.1	60	38.91
$C_{0.5_30}$	•	•	0.5	30	38.84**	C_{B1}	•		–	–	38.92
$C_{0.7_120}$	•	•	0.7	120	38.84**	$C_{0.1_20}$	•	•	0.1	20	38.92
$C_{0.7_20}$	•	•	0.7	20	38.85*	$C_{0.3_10}$	•	•	0.3	10	38.94
$C_{0.7_40}$	•	•	0.7	40	38.85*	$C_{0.7_10}$	•	•	0.7	10	38.96
$C_{0.5_10}$	•	•	0.5	10	38.86*	$C_{0.1_10}$	•	•	0.1	10	38.96
$C_{0.7_30}$	•	•	0.7	30	38.86	C_{B0}			–	–	39.92***
$C_{0.7_60}$	•	•	0.7	60	38.86*	$C_{0.9_120}$	•	•	0.9	120	40.38***
$C_{0.3_120}$	•	•	0.3	120	38.87	$C_{0.9_60}$	•	•	0.9	60	40.49***
$C_{0.3_30}$	•	•	0.3	30	38.88	$C_{0.9_30}$	•	•	0.9	30	40.62***
$C_{0.3_20}$	•	•	0.3	20	38.88	$C_{0.9_40}$	•	•	0.9	40	40.63***
$C_{0.3_40}$	•	•	0.3	40	38.88	$C_{0.9_20}$	•	•	0.9	20	40.82***
$C_{0.1_40}$	•	•	0.1	40	38.9	$C_{0.9_10}$	•	•	0.9	10	41.19***

Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

Ackermann C, Rieck J (2023) A novel repositioning approach and analysis for dynamic ride-hailing problems. EURO Journal on Transportation and Logistics 12:100109. https://doi.org/10.1016/j.ejtl.2023.100109

Agussurja L, Cheng SF, Lau HC (2019) A state aggregation approach for stochastic multiperiod last-mile ride-sharing problems. Transp Sci 53(1):148–166

Alonso-Mora J, Samaranayake S, Wallar A, Frazzoli E, Rus D (2017) On-demand high-capacity ride-sharing via dynamic trip-vehicle assignment. Proc Natl Acad Sci 114(3):462–467

Alonso-Mora J, Wallar A, Rus D (2017b) Predictive routing for autonomous mobility-on-demand systems with ride-sharing. In: 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), IEEE, pp 3583–3590

Anzenhofer F, Fleckenstein D, Klein R, Steinhardt C (2024) Sustainable dynamic pricing for rural shared mobility-on-demand systems. Available at SSRN 5077836

Anzenhofer F, Schmidbaur S, Klein R, Steinhardt C (2025) The potential of governmental regulation on shared mobility-on-demand systems. Transportation Research Part A Policy and Practice 192:104360

Attanasio A, Cordeau JF, Ghiani G, Laporte G (2004) Parallel tabu search heuristics for the dynamic multi-vehicle dial-a-ride problem. Parallel Comput 30(3):377–387

Avraham E, Raviv T (2021) The steady-state mobile personnel booking problem. Transportation Research Part B Methodological 154:266–288

Berbeglia G, Cordeau JF, Laporte G (2010) Dynamic pickup and delivery problems. Eur J Oper Res 202(1):8–15. https://doi.org/10.1016/j.ejor.2009.04.024

Berthold L, Fliedner M, Schulz A (2025) A shift scheduling model for ridepooling services. OR Spectrum 47:349–373

Brinkmann J, Ulmer MW, Mattfeld DC (2019) Dynamic lookahead policies for stochastic-dynamic inventory routing in bike sharing systems. Computers & Operations Research 106:260–279

Coslovich L, Pesenti R, Ukovich W (2006) A two-phase insertion technique of unexpected customers for a dynamic dial-a-ride problem. Eur J Oper Res 175(3):1605–161. https://doi.org/10.1016/j.ejor.2005.02.038

Fagnant DJ, Kockelman KM (2014) The travel and environmental implications of shared autonomous vehicles, using agent-based model scenarios. Transportation Research Part C Emerging Technologies 40:1–13. https://doi.org/10.1016/j.trc.2013.12.001

Fleckenstein D, Klein R, Steinhardt C (2023) Recent advances in integrating demand management and vehicle routing: A methodological review. Eur J Oper Res 306(2):499–518

Gaul D, Klamroth K, Stiglmayr M (2021) Solving the Dynamic Dial-a-Ride Problem Using a Rolling-Horizon Event-Based Graph. In: Müller-Hannemann M, Perea F (eds) 21st Symposium on Algorithmic Approaches for Transportation Modelling, Optimization, and Systems (ATMOS 2021), Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, Open Access Series in Informatics (OASIcs), vol 96, pp 8:1–8:1https://doi.org/10.4230/OASIcs.ATMOS.2021.8

Gaul D, Klamroth K, Pfeiffer C, Stiglmayr M, Schulz A (2025) A tight formulation for the dial-a-ride problem. Eur J Oper Res 321(2):363–382

Geržinič N, Cats O, van Oort N, Hoogendoorn-Lanser S, Bierlaire M, Hoogendoorn S (2023) An instance-based learning approach for evaluating the perception of ride-hailing waiting time variability. Travel Behaviour and Society 33:100616

Goodson JC, Ohlmann JW, Thomas BW (2013) Rollout policies for dynamic solutions to the multivehicle routing problem with stochastic demand and duration limits. Oper Res 61(1):138–154

Gschwind T, Drexl M (2019) Adaptive large neighborhood search with a constant-time feasibility test for the dial-a-ride problem. Transp Sci 53(2):480–491

Haferkamp J (2024) Design of multi-optional pickup time offers in ride-sharing systems. EURO Journal on Transportation and Logistics 13:100134
Häll CH, Lundgren JT, Voß S (2015) Evaluating the performance of a dial-a-ride service using simulation. Public Transport 7(2):139–157. https://doi.org/10.1007/s12469-015-0101-z
Hanne T, Melo T, Nickel S (2009) Bringing robustness to patient flow management through optimized patient transports in hospitals. INFORMS Journal on Applied Analytics 39(3):241–25. https://doi.org/10.1287/inte.1080.0379
Heitmann RJO, Soeffker N, Ulmer MW, Mattfeld DC (2023) Combining value function approximation and multiple scenario approach for the effective management of ride-hailing services. EURO Journal on Transportation and Logistics 12:100104
Ho SC, Szeto WY, Kuo YH, Leung JM, Petering M, Tou TW (2018) A survey of dial-a-ride problems: Literature review and recent developments. Transportation Research Part B Methodological 111:395–421. https://doi.org/10.1016/j.trb.2018.02.001
Hosni H, Naoum-Sawaya J, Artail H (2014) The shared-taxi problem: Formulation and solution methods. Transportation Research Part B Methodological 70:303–318
Hungerländer P, Rendl A, Truden C (2017) On the slot optimization problem in on-line vehicle routing. Transportation Research Procedia 27:492–499
Ichoua S, Gendreau M, Potvin JY (2006) Exploiting knowledge about future demands for real-time vehicle dispatching. Transp Sci 40(2):211–22. https://doi.org/10.1287/trsc.1050.0114
Illgen S, Höck M (2019) Literature review of the vehicle relocation problem in one-way car sharing networks. Transportation Research Part B Methodological 120:193–20. https://doi.org/10.1016/j.trb.2018.12.006
ioki GmbH (2024) ioki — digital public transport. https://ioki.com/en/home/ (accessed January 18, 2024)
Chu JCFYL, Lin HC, Yu YH (2023) Dynamic repositioning problem of dockless electric scooter sharing systems. Transportation Letters 15(9):1066–108. https://doi.org/10.1080/19427867.2022.2129209
Jaw JJ, Odoni AR, Psaraftis HN, Wilson NH (1986) A heuristic algorithm for the multi-vehicle advance request dial-a-ride problem with time windows. Transportation Research Part B Methodological 20(3):243–257. https://doi.org/10.1016/0191-2615(86)90020-2
Kaufman L, Rousseeuw PJ (1990) Finding Groups in Data: An Introduction to Cluster Analysis. Wiley Series in Probability and Statistics Wiley. https://doi.org/10.1002/SERIES1345
Ke J, Xiao F, Yang H, Ye J (2020) Learning to delay in ride-sourcing systems: A multi-agent deep reinforcement learning framework. IEEE Trans Knowl Data Eng 34(5):2280–2292
Klein R, Neugebauer M, Ratkovitch D, Steinhardt C (2019) Differentiated time slot pricing under routing considerations in attended home delivery. Transp Sci 53(1):236–255
Klein V, Steinhardt C (2023) Dynamic demand management and online tour planning for same-day delivery. Eur J Oper Res 307(2):860–886
Koch S, Klein R (2020) Route-based approximate dynamic programming for dynamic pricing in attended home delivery. Eur J Oper Res 287(2):633–652
Lang MA, Cleophas C, Ehmke JF (2021) Multi-criteria decision making in dynamic slotting for attended home deliveries. Omega 102:102305
Liu Y, Bansal P, Daziano R, Samaranayake S (2019) A framework to integrate mode choice in the design of mobility-on-demand systems. Transportation Research Part C Emerging Technologies 105:648–665
Luo Y, Schonfeld P (2011) Online rejected-reinsertion heuristics for dynamic multivehicle dial-a-ride problem. Transportation Research Record Journal of the Transportation Research Board 2218(1):59–6. https://doi.org/10.3141/2218-07
Ma H, Fang F, Parkes DC (2022) Spatio-temporal pricing for ridesharing platforms. Oper Res 70(2):1025–1041
Mackert J (2019) Choice-based dynamic time slot management in attended home delivery. Computers & Industrial Engineering 129:333–345
Madsen OBG, Ravn HF, Rygaard JM (1995) A heuristic algorithm for a dial-a-ride problem with time windows, multiple capacities, and multiple objectives. Ann Oper Res 60(1):193–20. https://doi.org/10.1007/BF02031946
MOIA GmbH (2024) Ride the mobility of tomorrow. https://www.moia.io/en (accessed January 18, 2024)
Núñez A, Cortés CE, Sáez D, de Schutter B, Gendreau M (2014) Multiobjective model predictive control for dynamic pickup and delivery problems. Control Eng Pract 32:73–86. https://doi.org/10.1016/j.conengprac.2014.07.004
Padam Mobility (2024) Demand responsive transport — reinvented. https://padam-mobility.com/en/ (accessed January 18, 2024)

Pfeiffer C, Schulz A (2022) An ALNS algorithm for the static dial-a-ride problem with ride and waiting time minimization. OR Spectrum 44:87–119
Pfeiffer C, Schulz A (2022b) A new lower bound for the static dial-a-ride problem with ride and waiting time minimization. In: International Conference on Dynamics in Logistics, Springer, pp 231–243
Powell WB (2019) Reinforcement Learning and Stochastic Optimization – A unified framework for sequential decisions. John Wiley & Sons, Hoboken, New Jersey
Powell WB (2022) Designing lookahead policies for sequential decision problems in transportation and logistics. IEEE Open Journal of Intelligent Transportation Systems 3:313–327
Quadrifoglio L, Dessouky MM, Ordóñez F (2008) A simulation study of demand responsive transit system design. Transportation Research Part A Policy and Practice 42(4):718–73. https://doi.org/10.1016/j.tra.2008.01.018
Savelsbergh MW (1992) The vehicle routing problem with time windows: Minimizing route duration. ORSA J Comput 4(2):146–154
Sayarshad HR, Gao HO (2018) A scalable non-myopic dynamic dial-a-ride and pricing problem for competitive on-demand mobility systems. Transportation Research Part C Emerging Technologies 91:192–208
Schilde M, Doerner KF, Hartl RF (2011) Metaheuristics for the dynamic stochastic dial-a-ride problem with expected return transports. Computers & Operations Research 38(12):1719–173. https://doi.org/10.1016/j.cor.2011.02.006
Schmid V (2012) Solving the dynamic ambulance relocation and dispatching problem using approximate dynamic programming. Eur J Oper Res 219(3):611–621
Schulz A, Pfeiffer C (2024) A branch-and-cut algorithm for the dial-a-ride problem with incompatible customer types. Transportation Research Part E Logistics and Transportation Review 181:103394
Schulz A, Pfeiffer C (2024) Using fixed paths to improve branch-and-cut algorithms for precedence-constrained routing problems. Eur J Oper Res 312(2):456–472
Shah S, Lowalekar M, Varakantham P (2020) Neural approximate dynamic programming for on-demand ride-pooling. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol 34(01), pp 507–515
Sheridan PK, Gluck E, Guan Q, Pickles T, Balcıog B, Benhabib B (2013) The dynamic nearest neighbor policy for the multi-vehicle pick-up and delivery problem. Transportation Research Part A Policy and Practice 49:178–194
Soeffker N, Ulmer MW, Mattfeld DC (2022) Stochastic dynamic vehicle routing in the light of prescriptive analytics: A review. Eur J Oper Res 298(3):801–820
Souza ALS, Bernardo M, Penna PHV, Pannek J, Souza MJF (2022) Bi-objective optimization model for the heterogeneous dynamic dial-a-ride problem with no rejects. Optimization Letters 16(1):355–374. https://doi.org/10.1007/s11590-020-01698-6
Swihart MR, Papastavrou JD (1999) A stochastic and dynamic model for the single-vehicle pick-up and delivery problem. Eur J Oper Res 114(3):447–46, https://doi.org/10.1016/S0377-2217(98)00260-4, https://www.sciencedirect.com/science/article/pii/S0377221798002604
Tafreshian A, Abdolmaleki M, Masoud N, Wang H (2021) Proactive shuttle dispatching in large-scale dynamic dial-a-ride systems. Transportation Research Part B Methodological 150:227–25. https://doi.org/10.1016/j.trb.2021.06.002
Ulmer MW (2020) Dynamic pricing and routing for same-day delivery. Transp Sci 54(4):1016–1033
Ulmer MW, Thomas BW (2020) Meso-parametric value function approximation for dynamic customer acceptances in delivery routing. Eur J Oper Res 285(1):183–195
Ulmer MW, Goodson JC, Mattfeld DC, Hennig M (2019) Offline-online approximate dynamic programming for dynamic vehicle routing with stochastic requests. Transp Sci 53(1):185–202
Ulmer MW, Goodson JC, Mattfeld DC, Thomas BW (2020) On modeling stochastic dynamic vehicle routing problems. EURO Journal on Transportation and Logistics 9(2):100008
Vallée S, Oulamara A, Cherif-Khettaf WR (2017) Maximizing the number of served requests in an online shared transport system by solving a dynamic DARP. In: International Conference on Computational Logistics, Springer, pp 64–78
Vallée S, Oulamara A, Cherif-Khettaf WR (2019) Reinsertion algorithm based on destroy and repair operators for dynamic dial a ride problems. In: International Conference on Computational Science, Springer, pp 81–95
Vallée S, Oulamara A, Cherif-Khettaf WR (2020) New online reinsertion approaches for a dynamic dial-a-ride problem. Journal of Computational Science 47:10119. https://doi.org/10.1016/j.jocs.2020.101199

Wang X, Liu W, Yang H, Wang D, Ye J (2019) Customer behavioural modelling of order cancellation in coupled ride-sourcing and taxi markets. Transportation Research Procedia 38:853–873

Waßmuth K, Köhler C, Agatz N, Fleischmann M (2023) Demand management for attended home delivery-a literature review. Eur J Oper Res 311(3):801–815

Xiang Z, Chu C, Chen H (2008) The study of a dynamic dial-a-ride problem under time-dependent and stochastic environments. Eur J Oper Res 185(2):534–55. https://doi.org/10.1016/j.ejor.2007.01.007

Yang X, Strauss AK, Currie CS, Eglese R (2016) Choice-based demand management and vehicle routing in e-fulfillment. Transp Sci 50(2):473–488

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.